KUZLAB - LLM PROJECT

- KUZ LLM DUO - RUN AWAY - #01
- KUZ LLM DUO - RUN AWAY - #02
- KUZ LLM DUO - RUN AWAY - #03
- KUZ LLM DUO - RUN AWAY - #04
- KUZ LLM DUO - APPLICATION SYNOPTIC

KUZ LLM DUO - RUN AWAY - #01

`LLM - DUO - RUN AWAY - #01`

`WHAT'S INSIDE THE BLACK BOX ?`

`INTRODUCTION`

Within THE KUZ NETWORK laboratory, a new experiment will soon be set up around the local execution of open-source language models. This project, called Local LLM Duo, will consist of running two distinct models on two different machines and then automating their exchanges through a central orchestrator.

The goal will not be to produce a simple demonstration effect. The project will primarily serve as a working platform to study how several open-source components can be assembled into a coherent local environment --> Linux system, GPU drivers, inference engine, LLM models, network API, and orchestration logic.

The real interest of the lab will lie in this integration layer, meaning the way multiple technical components will be connected cleanly in order to create a continuous inference loop between two model instances.

The installation is done. This document therefore presents the planned architecture, the tools that will be used, and the way the lab will be built from a technical perspective.

`OVERALL ARCHITECTURE`

The lab will rely on a simple distributed architecture built around two physical nodes connected over the local network.

The first machine will act as the main node. It will host the orchestrator, the first language model, and the control interface for the experiment. This machine will be used to start sessions, monitor exchanges, and store technical logs.

The second machine will run another language model. It will not control the session itself, but will expose an inference endpoint over the network so that the orchestrator can send requests to it and retrieve its responses.

The planned workflow will be as follows --> an initial topic will be injected into the first model, the generated answer will be captured by the orchestrator, then forwarded to the second model.

The second model's reply will then be sent back to the first one, and the loop will continue according to the parameters defined when the session starts.

The system will therefore rely on a chain of requests controlled by the orchestrator, with preservation of the conversational context and management of turn order.

`HARDWARE INFRASTRUCTURE`

Machine A will be the main workstation of the lab. It will be equipped with an Intel i9 processor, 32 GB RAM, and an NVIDIA RTX 5060 8G VRAM. This machine will have a dual role: it will run the first model and also host the entire control logic of the experiment.

Machine B will serve as the second inference node. It will be based on an Intel i5, 32 GB of RAM, and an NVIDIA RTX 3050 4G VRAM. It will host the second model and answer requests sent from the main machine.

This hardware separation will make it possible to test a real distributed setup, with two independent execution environments, two separate GPU workloads, and actual inter-machine communication.

This matters because the experiment will not only focus on the responses produced by the models, but also on the way a lightweight multi-node system will be built around them.

`OPENS-SOURCE TOOLS AND COMPONENTS`

The lab will rely on a set of relatively lightweight open-source tools suited to this type of experimentation.

The system foundation will most likely be built on Linux, using a base such as Debian, in order to provide a stable environment, straightforward administration, and compatibility with the GPU tooling required for local inference.

For the model execution layer, the planned choice will be Llama.cpp. This tool will make it possible to run language models locally, manage them as services, and, most importantly, expose a simple HTTP API that can easily be used from a Python script.

This point will be central to the project, since the orchestrator will rely on this API to manage the exchanges between the two machines.

As for the models themselves, several open-source families may be tested depending on the available resources and the results sought. The lab will probably begin with models from the Open-Source Models family, then possibly compare their behavior with other models available in the Llama.cpp ecosystem.

The goal will not be to lock the project immediately to a single model, but rather to create a flexible technical base that will make it easy to switch engines or models over time.

The overall design will therefore rely on four clearly separated layers: the Linux system, the NVIDIA GPU stack, the local inference engine, and the orchestration program.

`TECHNICAL DEPLOYMENT`

The lab deployment will begin with the preparation of both machines. The operating system will first need to be installed, the base updates applied, the local network configured, and communication between both nodes verified.

This stage will include address resolution, connectivity testing, and basic validation of network exchanges between the main machine and the second node.

Once the system base is ready, the priority will be the GPU layer setup. NVIDIA drivers will be installed on both machines, and the CUDA stack compatible with the selected environment will be verified.

At this stage, the goal will not yet be to run the models, but to confirm that the GPUs will be correctly recognized by the system and usable by the inference services.

The first checks will therefore focus on hardware detection, driver versions, and the general stability of the compute environment.

After that phase, the lab will move on to installing the inference engine. Llama.cpp will be deployed on each machine and started locally as a service. It will then be necessary to verify that each node can correctly answer local API requests. This stage will confirm that a simple prompt can be sent to the model and that a structured response can be retrieved without service-side errors.

Once Llama.cpp is operational, the first models will be downloaded and loaded onto each machine. This is where the lab will begin testing compatibility between the size of the chosen models, the available memory, and the actual GPU performance.

It will probably be necessary to adjust the initial choices depending on memory usage, observed inference speed, and response stability. The lab will therefore be built progressively, prioritizing reliable operation before any advanced optimization.

When both machines are able to answer inference calls independently, the next step will be to connect both services within the same application logic.

`ORCHESTRATOR DESIGN`

`WEB CONSOLE ORCHESTRATOR - APPLICATION LAYER`

The project integrates a web-based control console developed in PHP. This layer provides real-time interaction with the orchestrator while keeping execution logic isolated in Python.

The web application acts strictly as a control and visualization interface.

`Architecture -->`

- PHP frontend/backend for UI and request handling\
- Python orchestrator for execution logic\
- Llama.cpp APIs for inference

Communication is handled through HTTP requests between PHP and the orchestrator.

`Application structure -->`

- /public → entry point\
- /assets → UI styling\
- /api → orchestrator calls\
- /core → bridge with Python\
- /config → profiles and parameters

The interface is divided into two sections.

`STATUS -->`

- node availability\
- hardware metrics\
- API connectivity

`DISCUSSION -->`

- prompt input\
- run controls\
- profiles\
- live transcript

`Execution flow -->`

- user submits prompt\
- PHP sends request\
- orchestrator runs loop\
- responses returned and displayed

The interface allows parameter control (turns, temperature, roles) and profile reuse via JSON/YAML.

All sessions are logged, while the UI provides real-time visualization.

This layer turns the project into a complete operational system combining orchestration and user interaction.

The orchestrator will be the software core of the project. It will not be a simple test script, but a lightweight control layer responsible for message routing, context construction, and global session management.

The natural choice will be Python, since this language will make it possible to build clear, modular logic that can easily evolve over time. The orchestrator will most likely use a standard HTTP library to communicate with the Llama.cpp APIs running on both machines.

It will also include an internal data structure responsible for storing the conversation history, along with a prompt-formatting layer so that each model receives usable context.

The program will begin by loading a configuration defining the essential parameters --> IP addresses or DNS names of both nodes, ports used by the Llama.cpp services, model names, inference settings, maximum number of turns, and possibly dedicated system instructions for each role.

This configuration phase will be important because it will make it possible to keep the application logic clearly separated from the experiment parameters.

Once initialized, the orchestrator will inject a starting topic into model A. The response will be stored, formatted, and then forwarded to model B as a new context element. Model B will generate its own reply, which will in turn be recorded and sent back to model A.

This loop will repeat until a predefined stop condition is reached, such as a fixed number of iterations, a manual stop, or a context-length threshold.

A key technical point will be conversation context management. Since the models do not have persistent memory, the orchestrator will have to rebuild at each turn a prompt containing at least the initial topic, the latest exchanged messages, and, when required, a condensed version of the older parts. This means that the program will not merely pass text between two APIs: it will also perform actual contextual reconstruction.

As the discussion grows longer, the orchestrator will also have to handle another standard issue --> the maximum context size supported by the models. The planned design will therefore eventually include a context-reduction strategy.

This reduction may rely on partial truncation of older messages or on the generation of an intermediate summary that preserves the essential points of the exchange. This mechanism will be crucial in order to maintain long sessions without saturating the model's context window.

At the same time, the orchestrator will have to produce usable traceability. Each session should ideally be logged into a text or Markdown file with timestamps, identification of the speaking model, message content, and possibly technical metadata about each generation turn. This logging layer will be useful both for functional tracking and for debugging the environment.

In the longer term, the program may also include complementary features such as improved console rendering, separation between system prompts and user messages, or a configuration mode based on YAML or JSON files.

At first, however, the priority will remain the creation of a stable, readable, and reproducible loop between the two nodes.

SOFTWARE DEPLOYMENT SEQUENCE

The lab will probably be built in successive stages. The first phase will consist in validating each machine independently. stable system, detected GPU, functional Llama.cpp service, loaded model, and correct response to a local call. Until this base is fully under control, moving on to inter-machine orchestration will not be relevant.

The second phase will consist in validating remote communication. From the main machine, it will be necessary to confirm that an HTTP call to Machine B can successfully retrieve a response from the remote model. This step will help isolate network, firewall, service, or request-format issues before the full dialogue logic is introduced.

The third phase will be the development of the first functional orchestrator. It will initially remain deliberately simple. Sending a starting topic, alternating responses, displaying the exchange in the console, and storing a minimal history. Once this base is validated, the code will be extended with better context handling, cleaner logs, adjustable parameters, and more robust behavior in case of request errors or empty responses.

The lab will therefore not be built as a monolithic block, but as a set of technical building blocks validated one by one. This method will make it possible to keep a clear understanding of the system, identify friction points quickly, and evolve the project cleanly.

PROJECT PURPOSE

Through Local LLM Duo, the KUZ NETWORK laboratory will set up an experimental environment focused on local open-source LLMs, GPU-based inference, and their orchestration within a lightweight distributed architecture.

The interest of the project will lie as much in the models themselves as in the technical setup built around them. The real subject of the lab will be the construction of a coherent, reproducible, and fully controlled small-scale system capable of making two model instances interact using only open-source tools and local infrastructure.

In that sense, Local LLM Duo will be less a simple demonstration than a practical testbed for exploring the behavior of open-source models, context management, the stability of a local inference engine, and the orchestration logic required to coordinate multiple nodes inside the same application loop.

API ENDPOINTS - ORCHESTRATOR

The orchestrator exposes a minimal HTTP interface to allow interaction with the web console.

`Example endpoints -->`

POST /run
Triggers a new orchestration session.

`Payload -->` { "prompt": "initial topic", "max_turns": 10, "temperature": 0.7, "profile": "default" }

`Response -->` { "status": "running", "session_id": "uuid" }

GET /status/{session_id}
Returns current execution state.

GET /transcript/{session_id}
Returns full conversation history.

DATA FLOW SCHEMA

User → Web UI (PHP) → API call → Python Orchestrator
→ Model A (Machine A)
→ Orchestrator
→ Model B (Machine B)
→ Orchestrator loop
→ PHP → UI rendering

SESSION FILE STRUCTURE

Each session can be stored as JSON -->

{ "session_id": "...", "timestamp": "...", "turns": [{ "speaker": "model_a", "content": "...", "time": "..." }, { "speaker": "model_b", "content": "...", "time": "..." }] }

This enables replay, audit, and debugging.

FUTURE EXTENSIONS

Planned improvements -->

- WebSocket streaming for real-time tokens\
- Multi-model orchestration (more than 2 nodes)\
- UI-based orchestrator builder\
- Advanced context compression strategies\
- Metrics dashboard (latency, tokens, GPU usage)

The system is designed to evolve without modifying core architecture.

KUSANAGI8200 - THE KUZ NETWORK - @2026

KUZ LLM DUO - RUN AWAY - #02

`LLM - DUO - RUN AWAY - #02`

`INSTALLING THE FIRST MACHINE - FHC2 / KUZAI NODE`

The first node of the LLM Duo project was installed and prepared as a dedicated local inference machine for THE KUZ NETWORK lab. The goal of this phase was not only to get a model running, but to turn the machine into a clean, reproducible, headless Linux node ready to host a local open-source LLM and later integrate into a two-node orchestration workflow.

`1/ HARDWARE CHECK`

The machine used for this first node is FHC2, an Acer Nitro ANV15-52 running Ubuntu 24.04.4 LTS with kernel 6.17.0-19-generic. It is equipped with an Intel Core i9-13900H, 32 GiB of RAM, and an NVIDIA RTX 5060 Laptop GPU. Storage is provided by a 1 TB Kingston NVMe drive, with the system currently installed on a single main ext4 partition. The machine boots in UEFI mode.

```
hostnamectl
cat /etc/os-release
uname -a
```

```
echo "===== CPU ====="
lscpu | egrep 'Model name|Socket|Thread|Core|CPU\(s\)'

echo "===== RAM ====="
free -h
grep MemTotal /proc/meminfo

echo "===== GPU ====="
lspci -nnk | grep -A3 -E 'VGA|3D|Display'

echo "===== DISKS / PARTITIONS ====="
lsblk -e7 -o NAME,SIZE,TYPE,FSTYPE,FSUSE%,MOUNTPOINTS,MODEL
blkid
df -hT
```

```
echo "===== BOOT MODE ====="
[ -d /sys/firmware/efi ] && echo UEFI || echo BIOS

echo "===== NETWORK ====="
ip -br a
ip r

echo "===== INSTALLED DESKTOP / DISPLAY STACK ====="
dpkg -l | egrep 'ubuntu-desktop|gnome-shell|gdm3|xorg|wayland|lightdm|sddm|network-manager'

echo "===== NVIDIA / NOUVEAU / CUDA ====="
dpkg -l | egrep 'nvidia|cuda|nouveau'
lsmod | egrep 'nvidia|nouveau'

lsblk -f
df -hT
blkid /dev/nvme0n1p2 /dev/nvme0n1p3 /dev/nvme0n1p4
```

`2/ SYSTEM CLEANUP`

The initial Ubuntu Desktop installation was converted into a server-style node. The graphical stack was disabled by switching the default target to multi-user.target, and the machine was kept reachable over SSH and Wi-Fi through NetworkManager. The purpose of this step was to remove the desktop overhead and keep only the components required for remote administration, networking, NVIDIA support, and local inference.

```
systemctl set-default multi-user.target
systemctl disable gdm3.service
reboot
```

```
systemctl get-default
systemctl is-enabled gdm3 || true
nmcli general status
ip -br a
nvidia-smi
```

```
apt-mark manual \
  openssh-server \
  network-manager \
```

```
  wpasupplicant \
  netplan.io \
  nvidia-driver-580-open \
  nvidia-utils-580 \
  nvidia-compute-utils-580 \
  linux-modules-nvidia-580-open-generic-hwe-24.04
```

```
apt -s purge \
  ubuntu-desktop-minimal \
  gdm3 \
  gnome-shell \
  gnome-shell-common \
  gnome-shell-extension-appindicator \
  gnome-shell-extension-desktop-icons-ng \
  gnome-shell-extension-ubuntu-dock \
  gnome-shell-extension-ubuntu-tiling-assistant \
  xorg \
  xserver-xorg \
  xserver-xorg-core \
  xserver-xorg-input-all \
  xserver-xorg-video-all \
  xwayland \
  yaru-theme-gnome-shell \
  network-manager-gnome \
  network-manager-openvpn-gnome \
  network-manager-pptp-gnome \
  xserver-xorg-video-amdgpu \
  xserver-xorg-video-ati \
  xserver-xorg-video-fbdev \
  xserver-xorg-video-intel \
  xserver-xorg-video-nouveau \
  xserver-xorg-video-qxl \
  xserver-xorg-video-radeon \
  xserver-xorg-video-vesa \
  xserver-xorg-video-vmware

apt -s autoremove --purge
```

```
dpkg -l | egrep 'ubuntu-desktop|gnome-shell|gdm3|xorg|xwayland' || true
systemctl get-default
```

```
nmcli general status
ip -br a
nvidia-smi
```

3/ HEADLESS HARDENING

Because this first node is a laptop-based machine, additional hardening was applied to avoid unwanted power-management behavior. Lid switch actions were disabled, idle-triggered actions were disabled, and sleep, suspend, hibernate, and hybrid-sleep targets were masked. The node was therefore turned into a more stable headless lab system suitable for long-running local inference sessions.

```
mkdir -p /etc/systemd/logind.conf.d
```

```
cat > /etc/systemd/logind.conf.d/headless.conf <<'EOF'
[Login]
HandleLidSwitch=ignore
HandleLidSwitchExternalPower=ignore
HandleLidSwitchDocked=ignore
IdleAction=ignore
EOF
```

```
systemctl mask sleep.target suspend.target hibernate.target hybrid-sleep.target
systemctl restart systemd-logind
```

```
snap list

snap remove firefox || true
snap remove snap-store || true
snap remove firmware-updater || true
snap remove gnome-42-2204 || true
snap remove gtk-common-themes || true
snap remove snapd-desktop-integration || true
```

```
apt purge -y \
  xorg-docs-core \
  nvidia-settings \
  screen-resolution-extra

apt autoremove --purge -y
```

```
apt clean
```

```
apt purge -y \
  gnome-bluetooth-3-common \
  gnome-bluetooth-sendto \
  gnome-control-center \
  gnome-control-center-data \
  gnome-control-center-faces \
  gnome-desktop3-data \
  gnome-keyring \
  gnome-keyring-pkcs11 \
  gnome-online-accounts \
  gnome-remote-desktop \
  gnome-settings-daemon \
  gnome-settings-daemon-common \
  gnome-user-docs \
  gnome-user-docs-fr \
  language-pack-gnome-fr \
  language-pack-gnome-fr-base \
  language-selector-gnome \
  libpam-gnome-keyring \
  pinentry-gnome3 \
  xdg-desktop-portal-gnome \
  snapd
```

```
apt autoremove --purge -y
apt clean
```

```
snap list || true
dpkg -l | egrep 'gnome|snapd|xorg|gdm' || true
nmcli general status
ip -br a
nvidia-smi
```

`4/ LLM NODE PREPARATION`

The machine was prepared with the NVIDIA 580 open driver stack, which successfully loaded on the RTX 5060 Laptop GPU. After that, the CUDA 13.2 toolkit was installed and validated with nvcc, confirming that the system was ready for CUDA-based compilation and inference workloads. This step was essential before building the LLM runtime itself.

```
apt update

apt install -y \
  build-essential \
  cmake \
  ninja-build \
  git \
  curl \
  wget \
  jq \
  ca-certificates \
  pkg-config \
  libssl-dev \
  python3 \
  python3-venv \
  python3-pip \
  pciutils
```

```
mkdir -p /opt/llm
mkdir -p /opt/src
mkdir -p /var/log/llm-duo
```

```
apt-mark hold \
  nvidia-driver-580-open \
  nvidia-utils-580 \
  nvidia-compute-utils-580 \
  linux-modules-nvidia-580-open-generic-hwe-24.04
```

```
wget -O /usr/share/keyrings/cuda-ubuntu2404-keyring.gpg \
  https://developer.download.nvidia.com/compute/cuda/repos/ubuntu2404/x86_64/cuda-ubuntu2404-keyring.gpg

wget -O /etc/apt/preferences.d/cuda-repository-pin-600 \
  https://developer.download.nvidia.com/compute/cuda/repos/ubuntu2404/x86_64/cuda-ubuntu2404.pin
```

```
cat > /etc/apt/sources.list.d/cuda-ubuntu2404.sources <<'EOF'
Types: deb
URIs: https://developer.download.nvidia.com/compute/cuda/repos/ubuntu2404/x86_64/
```

```
Suites: /
Signed-By: /usr/share/keyrings/cuda-ubuntu2404-keyring.gpg
EOF
```

```
apt update
apt-cache policy cuda-toolkit-13-2
apt install -y cuda-toolkit-13-2
```

```
cat > /etc/profile.d/cuda.sh <<'EOF'
export PATH=/usr/local/cuda/bin:$PATH
export LD_LIBRARY_PATH=/usr/local/cuda/lib64:$LD_LIBRARY_PATH
EOF

chmod 644 /etc/profile.d/cuda.sh
source /etc/profile.d/cuda.sh
```

```
which nvcc
nvcc --version
ls -ld /usr/local/cuda /usr/local/cuda-13.2
```

5/ CLONING AND COMPILATION OF LLAMA.CPP

The llama.cpp repository was cloned locally and compiled from source with CUDA support enabled using CMake and Ninja. The resulting build produced the expected binaries, including llama-cli and llama-server, and dynamic linking checks confirmed that CUDA and cuBLAS libraries were correctly resolved. This established the local inference engine for the node.

```
cd /opt/src

git clone https://github.com/ggml-org/llama.cpp
cd /opt/src/llama.cpp

cmake -S . -B build -G Ninja -DGGML_CUDA=ON
cmake --build build --config Release -j"$(nproc)"

ls -1 build/bin | grep '^llama-' || true

ldd build/bin/llama-cli | egrep 'cuda|cublas|cudart|stdc\+\+|libm|libpthread' || true
```

`6/ FIRST CLI TEST / LOADING MODEL = GEMMA-3-1B-IT-GGUF:Q4_K_M`

An initial validation was performed with Gemma 3 1B IT in GGUF format to confirm that the CUDA build and local inference path were functional. After successful CLI and API tests, the node was then reworked to better match the project goals by replacing the Google model with a fully open-source alternative.

```
cd /opt/src/llama.cpp
./build/bin/llama-cli -hf ggml-org/gemma-3-1b-it-GGUF:Q4_K_M
nvidia-smi
```

START LLAMA-SERVER

```
./build/bin/llama-server \
  -hf ggml-org/gemma-3-1b-it-GGUF:Q4_K_M \
  --host 0.0.0.0 \
  --port 8080
```

API TEST

```
ss -ltnp | grep 8080
curl -s http://127.0.0.1:8080/v1/models | jq
```

```
curl -s http://127.0.0.1:8080/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{
    "messages": [
      {"role": "user", "content": "Introduce yourself in two sentences."}
    ]
  }' | jq '.choices[0].message.content'
```

7/ OPEN-SOURCE MODEL CONFIGURATION

The final model retained for node A is Mistral-7B-Instruct-v0.3 Q4_K_M in GGUF format. The model file was stored under /opt/llm/models, and the serving layer was configured around it. This choice is more consistent with the direction of the LLM Duo project, which aims to build a local, open-source, reproducible two-node setup rather than rely on a model that does not fully match that framing.

```
systemctl disable --now llama-server-a.service 2>/dev/null || true
rm -f /etc/systemd/system/llama-server-a.service
systemctl daemon-reload
```

```
rm -f /opt/llm/models/ggml-org_gemma-3-1b-it-GGUF_gemma-3-1b-it-Q4_K_M.gguf
rm -f /root/.cache/llama.cpp/ggml-org_gemma-3-1b-it-GGUF_gemma-3-1b-it-Q4_K_M.gguf
rm -f /root/.cache/llama.cpp/ggml-org_gemma-3-1b-it-GGUF_preset.ini
rm -f /root/.cache/llama.cpp/ggml-org_gemma-3-1b-it-GGUF_gemma-3-1b-it-Q4_K_M.gguf.etag
rm -f /root/.cache/llama.cpp/manifest=ggml-org=gemma-3-1b-it-GGUF=Q4_K_M.json
```

```
mkdir -p /opt/llm/models
mkdir -p /opt/llm/run
mkdir -p /var/log/llm-duo

id -u llm >/dev/null 2>&1 || useradd -r -s /usr/sbin/nologin -d /opt/llm llm

chown -R llm:llm /opt/llm
chown -R llm:llm /var/log/llm-duo

ls -lh /root/.cache/llama.cpp
ls -ld /opt/llm /opt/llm/models /var/log/llm-duo
```

```
./build/bin/llama-cli -hf bartowski/Mistral-7B-Instruct-v0.3-GGUF:Mistral-7B-Instruct-v0.3-
Q4_K_M.gguf

ls -lh /root/.cache/llama.cpp | grep 'Mistral-7B-Instruct-v0.3'
```

```
cp -f /root/.cache/llama.cpp/bartowski_Mistral-7B-Instruct-v0.3-GGUF_Mistral-7B-Instruct-v0.3-
Q4_K_M.gguf /opt/llm/models/
chown llm:llm /opt/llm/models/bartowski_Mistral-7B-Instruct-v0.3-GGUF_Mistral-7B-Instruct-
v0.3-Q4_K_M.gguf
ls -lh /opt/llm/models
```

```
cat > /etc/systemd/system/llama-server-a.service <<'EOF'
[Unit]
Description=llama.cpp server - Node A
After=network-online.target
Wants=network-online.target

[Service]
Type=simple
User=llm
Group=llm
```

```
WorkingDirectory=/opt/src/llama.cpp
Environment="PATH=/usr/local/cuda/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:
/bin"
Environment="LD_LIBRARY_PATH=/usr/local/cuda/lib64"
ExecStart=/opt/src/llama.cpp/build/bin/llama-server \
  -m /opt/llm/models/bartowski_Mistral-7B-Instruct-v0.3-GGUF_Mistral-7B-Instruct-v0.3-
Q4_K_M.gguf \
  --host 0.0.0.0 \
  --port 8080
Restart=always
RestartSec=5
NoNewPrivileges=true
PrivateTmp=true
ProtectSystem=full
ProtectHome=true
ReadWritePaths=/opt/llm /var/log/llm-duo

[Install]
WantedBy=multi-user.target
EOF
```

```
systemctl daemon-reload
systemctl enable --now llama-server-a.service

systemctl status llama-server-a.service --no-pager -l
ss -ltnp | grep 8080
```

```
curl -s http://127.0.0.1:8080/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{
    "messages": [
      {"role": "user", "content": "Introduce yourself in two sentences and confirm that you
are a local open-source model."}
    ]
  }' | jq '.choices[0].message.content'
```

8/ ORCHESTRATOR ENVIRONMENT PREPARATION

```
mkdir -p /opt/llm/orchestrator
python3 -m venv /opt/llm/orchestrator/venv
/opt/llm/orchestrator/venv/bin/pip install --upgrade pip requests
```

```
cat > /opt/llm/orchestrator/test_node_a.py <<'EOF'
#!/opt/llm/orchestrator/venv/bin/python3

import json
import sys
from typing import Any

import requests

URL = "http://127.0.0.1:8080/v1/chat/completions"

payload: dict[str, Any] = {
    "messages": [
        {
            "role": "user",
            "content": "Reply in one short sentence. Confirm that node A of the LLM Duo
project is working locally."
        }
    ]
}

def main() -> int:
    try:
        response = requests.post(URL, json=payload, timeout=120)
        response.raise_for_status()
        data = response.json()
        content = data["choices"][0]["message"]["content"]
        print(content)
        return 0
    except Exception as exc:
        print(f"ERROR: {exc}", file=sys.stderr)
        return 1

if __name__ == "__main__":
    raise SystemExit(main())
EOF
```

```
chmod 755 /opt/llm/orchestrator/test_node_a.py
/opt/llm/orchestrator/test_node_a.py
```

9/ LOCAL ORCHESTRATOR VALIDATION

```
cat > /opt/llm/orchestrator/duo_loop_local.py <<'EOF'
#!/opt/llm/orchestrator/venv/bin/python3

import argparse
import json
import sys
import time
from datetime import datetime
from pathlib import Path
from typing import Any

import requests



DEFAULT_URL = "http://127.0.0.1:8080/v1/chat/completions"



ROLE_A_SYSTEM = """You are Model A of the LLM Duo project.
Role: proposal, structuring, technical framing.
You formulate initial ideas, hypotheses, architecture options, and deployment choices.
Expected response: clear, concise, actionable, technically oriented.
Avoid long unnecessary introductions.
"""

ROLE_B_SYSTEM = """You are Model B of the LLM Duo project.
Role: critical analysis, improvement, technical review.
You challenge hypotheses, correct inaccuracies, add missing points, and propose concrete
improvements.
Expected response: clear, concise, actionable, technically oriented.
Avoid long unnecessary introductions.
"""



def build_prompt(
```

```
    role_prompt: str,
    topic: str,
    history: list[dict[str, str]],
    incoming_text: str | None,
) -> str:
    parts: list[str] = []

    parts.append("PROJECT: LLM Duo")
    parts.append("")
    parts.append("ROLE INSTRUCTION")
    parts.append(role_prompt.strip())
    parts.append("")
    parts.append("INITIAL TOPIC")
    parts.append(topic.strip())
    parts.append("")

    if history:
        parts.append("RECENT HISTORY")
        for item in history[-8:]:
            parts.append(f"{item['speaker']}:")
            parts.append(item["content"].strip())
            parts.append("")

    if incoming_text:
        parts.append("LAST MESSAGE FROM THE OTHER MODEL")
        parts.append(incoming_text.strip())
        parts.append("")
        parts.append(
            "TASK: reply to the last message while taking the initial topic and the history
into account. "
            "Provide a directly useful response."
        )
    else:
        parts.append(
            "TASK: this is the first turn. "
            "Start analyzing the topic with a directly actionable response."
        )

    return "\n".join(parts).strip()
```

```
def query_model(
    url: str,
    prompt: str,
    timeout: int = 300,
) -> str:
    payload: dict[str, Any] = {
        "messages": [
            {
                "role": "user",
                "content": prompt,
            }
        ],
        "temperature": 0.7,
    }

    response = requests.post(url, json=payload, timeout=timeout)
    response.raise_for_status()

    data = response.json()
    return data["choices"][0]["message"]["content"].strip()


def write_json(path: Path, obj: Any) -> None:
    path.write_text(
        json.dumps(obj, ensure_ascii=False, indent=2) + "\n",
        encoding="utf-8",
    )


def write_markdown(path: Path, topic: str, transcript: list[dict[str, Any]]) -> None:
    lines: list[str] = []
    lines.append("# LLM Duo Local Test")
    lines.append("")
    lines.append(f"**Topic**: {topic}")
    lines.append("")

    for item in transcript:
        lines.append(f"## Turn {item['turn']} — {item['speaker']}")
        lines.append("")
```

```
        lines.append(item["content"])
        lines.append("")

    path.write_text("\n".join(lines), encoding="utf-8")


def main() -> int:
    parser = argparse.ArgumentParser(description="LLM Duo local loop test on Node A")
    parser.add_argument(
        "--url",
        default=DEFAULT_URL,
        help="OpenAI-compatible local endpoint URL",
    )
    parser.add_argument(
        "--topic",
        required=True,
        help="Initial discussion topic",
    )
    parser.add_argument(
        "--turns",
        type=int,
        default=6,
        help="Total number of turns",
    )
    parser.add_argument(
        "--delay",
        type=float,
        default=1.0,
        help="Delay between turns in seconds",
    )
    parser.add_argument(
        "--output-dir",
        default="/opt/llm/orchestrator/runs",
        help="Directory for logs and transcripts",
    )

    args = parser.parse_args()

    output_dir = Path(args.output_dir)
    output_dir.mkdir(parents=True, exist_ok=True)
```

```
    ts = datetime.now().strftime("%Y%m%d-%H%M%S")
    run_dir = output_dir / f"run-{ts}"
    run_dir.mkdir(parents=True, exist_ok=True)

    history: list[dict[str, str]] = []
    transcript: list[dict[str, Any]] = []

    last_message: str | None = None

    speakers = [
        ("MODEL_A", ROLE_A_SYSTEM),
        ("MODEL_B", ROLE_B_SYSTEM),
    ]

    print(f"[INFO] Topic: {args.topic}")
    print(f"[INFO] Turns: {args.turns}")
    print(f"[INFO] Endpoint: {args.url}")
    print(f"[INFO] Run directory: {run_dir}")
    print("")

    for turn in range(1, args.turns + 1):
        speaker, system_prompt = speakers[(turn - 1) % 2]

        prompt = build_prompt(
            role_prompt=system_prompt,
            topic=args.topic,
            history=history,
            incoming_text=last_message,
        )

        print(f"===== TURN {turn} / {args.turns} — {speaker} =====")

        try:
            content = query_model(args.url, prompt)
        except Exception as exc:
            print(f"[ERROR] turn={turn} speaker={speaker} error={exc}", file=sys.stderr)
            return 1

        print(content)
```

```
        print("")

        item = {
            "turn": turn,
            "speaker": speaker,
            "content": content,
            "timestamp": datetime.now().isoformat(),
        }
        transcript.append(item)
        history.append({"speaker": speaker, "content": content})
        last_message = content

        time.sleep(args.delay)

    write_json(run_dir / "transcript.json", transcript)
    write_markdown(run_dir / "transcript.md", args.topic, transcript)

    print(f"[INFO] Transcript JSON: {run_dir / 'transcript.json'}")
    print(f"[INFO] Transcript MD:   {run_dir / 'transcript.md'}")

    return 0


if __name__ == "__main__":
    raise SystemExit(main())
EOF
```

```
chmod 755 /opt/llm/orchestrator/duo_loop_local.py
```

```
/opt/llm/orchestrator/duo_loop_local.py \
  --topic "Analyze the target architecture of the LLM Duo project and propose the first
technical deployment choices." \
  --turns 6
```

KUZ LLM DUO - RUN AWAY - #03

`LLM - DUO - RUN AWAY - #03`

`INSTALLING THE SECOND MACHINE - FHC / DARK-AI NODE`

Deploying the second node, the llama.cpp server, the orchestrator, and the first dialogues between two local models.

As part of the LOCAL LLM DUO PROJECT inside THE KUZ NETWORK LAB, I moved forward with the next stage of the setup with a clear objective 66> make two language models running locally on two different machines talk to each other while keeping full control over the infrastructure, the network, and the exchanges.

After preparing the first node, this new phase focused on installing the second node, deploying its inference engine, building the orchestrator that drives the exchange, and then validating the whole chain with the first automated dialogue tests.

`1/ INSTALLING THE SECOND NODE`

The first phase was dedicated to preparing the second machine in the lab. The goal was not just to run a model on it, but to turn it into a real compute node dedicated to the project.

The work began with a full hardware audit to confirm that the machine matched the intended role --> processor, memory, NVIDIA GPU, storage, installed operating system, and network state. Once this baseline was validated, the environment was cleaned up to remove components that were no longer useful in this context, especially the previous Ollama engine that was still present on the system.

This step mattered because it allowed the node to start again from a cleaner, more readable, and more coherent base aligned with the architecture chosen for the project.

`2/ INSTALLING THE LLAMA.CPP SERVER`

Once the node itself was ready, the next logical step was to install the inference engine selected for the lab --> llama.cpp.

The NVIDIA stack and the CUDA environment were checked first, then completed where needed so that the engine could be compiled with GPU support. The engine was then built locally with CUDA

acceleration, making it possible to run a lightweight local server fully aligned with an on-premise architecture.

A smaller open-source model was selected for this second node in order to preserve comfortable headroom on the available VRAM. The idea was not to push the machine to its absolute limit, but to keep the setup responsive, stable, and able to participate efficiently in the dialogue with the first model.

At the end of this phase, the second node was exposing a working local API through llama.cpp, just like the first one.

`3/ BUILDING THE ORCHESTRATOR`

Once both nodes were able to answer locally, the next essential component of the project had to be built: the orchestrator.

This orchestrator was developed as a Python script running from the main node. Its job is to manage the dialogue logic between the two models: entry point, turn alternation, forwarding of responses, output length limits, and transcript generation for each run.

The objective at this stage was not yet to build a finished application, but rather to create a simple, readable, modular base capable of running a real A ↔ B loop in a stable way.

This phase also made it possible to improve how instructions are sent to the models, so that the exchange would avoid overly generic drifts or responses that were unnecessarily long.

`4. TESTING THE DIALOGUE BETWEEN THE TWO MODELS`

Once the orchestrator was ready, several dialogue tests were launched between the two nodes.

This phase had a double purpose. On the one hand, it was necessary to validate the purely technical side: both servers respond correctly, the exchange flows properly, turn order is respected, and the outputs are archived. On the other hand, it was equally important to observe the conversational behavior of the models themselves.

The first tests showed that the infrastructure was working well, but they also made one thing very clear --> a dialogue between two models has to be strongly framed if one wants to avoid repetition, weak confirmation loops, or generic answers. This led to several changes in the orchestrator and in the system instructions, gradually pushing the setup toward a cleaner, more general, and more usable dialogue framework.

In other words, this phase validated not only the technical operation of the project, but also the first practical lessons on how to make two local AIs interact in a meaningful way.

`5. BUILDING A SMALL BASH APP TO LAUCH DISCUSSIONS`

To make the system easier to use, a small Bash launcher was then added to the project.

The idea was straightforward: avoid typing the full orchestration command manually every time a new test had to be launched. This interactive launcher now allows the user to fill only the useful variables, such as the opening prompt, the number of turns, or a few output limits, and then start the dialogue automatically between both models.

This Bash layer does not replace the Python orchestrator. It simply acts as a lightweight launch interface, useful for chaining tests more comfortably and making the whole project easier to operate in day-to-day lab work.

`6. CURRENT STATE OF THE PROJECT`

At this point, the LOCAL LLM DUO PROJECT has reached an important milestone.

Both nodes are now operational with their respective local models. The llama.cpp servers are functional on each machine. The orchestrator can run automated exchanges between the two models. And a small Bash tool now makes it easier to start test sessions without rebuilding each command manually.

The system is not yet a finished application, and that is precisely what makes this stage interesting. The lab is now entering a more experimental phase, where the goal is no longer only to install infrastructure, but to improve the actual quality of the exchange, the relevance of the model roles, and the overall fluidity of the orchestration.

So the project is progressing as planned --> first the technical foundation, then operational stability, and gradually after that, the quality of the dialogue itself.

`TEKNICAL VIEW + TESTS`

`1/ HARWARE CHECK`

```
hostnamectl
cat /etc/os-release
uname -a

echo
echo "===== CPU ====="
lscpu | egrep 'Model name|Socket|Thread|Core|CPU\(s\)'

echo
echo "===== RAM ====="
free -h
```

```
grep MemTotal /proc/meminfo

echo
echo "===== GPU ======"
lspci -nnk | grep -A3 -E 'VGA|3D|Display'

echo
echo "===== DISKS / PARTITIONS ====="
lsblk -e7 -o NAME,SIZE,TYPE,FSTYPE,FSUSE%,MOUNTPOINTS,MODEL
blkid
df -hT

echo
echo "===== BOOT MODE ====="
[ -d /sys/firmware/efi ] && echo UEFI || echo BIOS

echo
echo "===== NETWORK ====="
ip -br a
ip r

echo
echo "===== OLLAMA ====="
systemctl status ollama --no-pager -l 2>/dev/null || true
which ollama || true
dpkg -l | egrep 'ollama' || true
snap list 2>/dev/null | egrep 'ollama' || true

echo
echo "===== NVIDIA / CUDA ====="
nvidia-smi || true
which nvcc || true
nvcc --version || true
dpkg -l | egrep 'nvidia|cuda|nouveau' || true
lsmod | egrep 'nvidia|nouveau' || true
```

2/ DELETING OLLAMA ENGINE

```
systemctl disable --now ollama 2>/dev/null || true
pkill -f '/usr/local/bin/ollama' 2>/dev/null || true
```

```
rm -f /etc/systemd/system/ollama.service
rm -f /usr/local/bin/ollama

systemctl daemon-reload
systemctl reset-failed

id ollama 2>/dev/null || true
userdel ollama 2>/dev/null || true
groupdel ollama 2>/dev/null || true

rm -rf /usr/share/ollama
rm -rf /var/lib/ollama
rm -rf /var/log/ollama
rm -rf /etc/ollama
rm -rf /root/.ollama
rm -rf /home/elijah/.ollama 2>/dev/null || true
rm -rf /home/*/.ollama 2>/dev/null || true

echo
echo "===== VERIFY OLLAMA REMOVAL ======"
systemctl status ollama --no-pager -l 2>/dev/null || true
which ollama || true

find /etc/systemd/system /usr/local/bin /usr/share /var/lib /var/log /root /home -maxdepth 3
\( -iname '*ollama*' -o -iname '.ollama' \) 2>/dev/null
```

3/ PREPARE CUDA AND TOOLCHAIN

```
apt update

apt install -y \
  build-essential \
  cmake \
  ninja-build \
  git \
  curl \
  wget \
  jq \
  ca-certificates \
```

```
  pkg-config \
  libssl-dev \
  python3 \
  python3-venv \
  python3-pip \
  pciutils
```

```
apt-mark hold \
  nvidia-driver-575 \
  nvidia-utils-575 \
  nvidia-compute-utils-575 \
  nvidia-dkms-575
```

```
wget -O /usr/share/keyrings/cuda-archive-keyring.gpg \
  https://developer.download.nvidia.com/compute/cuda/repos/ubuntu2204/x86_64/cuda-archive-keyring.gpg

wget -O /etc/apt/preferences.d/cuda-repository-pin-600 \
  https://developer.download.nvidia.com/compute/cuda/repos/ubuntu2204/x86_64/cuda-ubuntu2204.pin
```

```
cat > /etc/apt/sources.list.d/cuda-ubuntu2204.list <<'EOF'
deb [signed-by=/usr/share/keyrings/cuda-archive-keyring.gpg] https://developer.download.nvidia.com/compute/cuda/repos/ubuntu2204/x86_64/ /
EOF

apt update

echo
echo "===== CUDA TOOLKIT CANDIDATES ======"
apt-cache policy cuda-toolkit-12-9 cuda-toolkit-13-2 cuda-toolkit-13

echo
echo "===== GPU ====="
nvidia-smi
```

4/ `SIMULATION AND INSTALL CUDA TOOLKIT`

```
apt install -y cuda-toolkit-12-9

cat > /etc/profile.d/cuda.sh <<'EOF'
export PATH=/usr/local/cuda/bin:$PATH
export LD_LIBRARY_PATH=/usr/local/cuda/lib64:$LD_LIBRARY_PATH
EOF

chmod 644 /etc/profile.d/cuda.sh
source /etc/profile.d/cuda.sh
```

```
echo
echo "===== CUDA TOOLKIT ======"
which nvcc
nvcc --version

echo
echo "===== CUDA PATHS ====="
ls -ld /usr/local/cuda /usr/local/cuda-12.9

echo
echo "===== GPU ====="
nvidia-smi
which nvcc
nvcc --version
ls -ld /usr/local/cuda /usr/local/cuda-12.9
nvidia-smi

echo
echo "===== CUDA LINKS ====="
ls -l /etc/alternatives/cuda
ls -l /usr/local/cuda
ls -l /usr/local/cuda/bin/nvcc /usr/local/cuda-12.9/bin/nvcc 2>/dev/null || true

echo
echo "===== CURRENT PATH ====="
echo "$PATH"
```

```
echo
echo "===== FORCE CUDA PATH ======"
export PATH=/usr/local/cuda/bin:$PATH
export LD_LIBRARY_PATH=/usr/local/cuda/lib64:$LD_LIBRARY_PATH
```

```
hash -r
which nvcc || true
/usr/local/cuda/bin/nvcc --version || true
nvcc --version || true
```

5/ PERSISTENCE CUDA + BUILD llama.cpp

```
cat > /etc/profile.d/cuda.sh <<'EOF'
export PATH=/usr/local/cuda/bin:$PATH
export LD_LIBRARY_PATH=/usr/local/cuda/lib64:$LD_LIBRARY_PATH
EOF
```

```
chmod 644 /etc/profile.d/cuda.sh
ln -sf /etc/profile.d/cuda.sh /etc/profile.d/zz-cuda.sh
```

```
apt update
apt install -y \
  build-essential \
  cmake \
  ninja-build \
  git \
  curl \
  wget \
  jq \
  ca-certificates \
  pkg-config \
  libssl-dev \
  python3 \
  python3-venv \
  python3-pip \
  pciutils
```

```
mkdir -p /opt/src
mkdir -p /opt/llm
mkdir -p /var/log/llm-duo

cd /opt/src
rm -rf /opt/src/llama.cpp

git clone https://github.com/ggml-org/llama.cpp
cd /opt/src/llama.cpp

export PATH=/usr/local/cuda/bin:$PATH
export LD_LIBRARY_PATH=/usr/local/cuda/lib64:$LD_LIBRARY_PATH
```

```
cmake -S . -B build -G Ninja -DGGML_CUDA=ON

cmake --build build --config Release -j"$(nproc)"
```

```
echo
echo "===== BINARIES ======"
ls -1 build/bin | grep '^llama-' || true

echo
echo "===== CUDA LINK CHECK ====="
ldd build/bin/llama-cli | egrep 'cuda|cublas|cudart|stdc\+\+|libm|libpthread' || true
```

6/ MODEL 3B CLI TEST

```
cd /opt/src/llama.cpp
export PATH=/usr/local/cuda/bin:$PATH
export LD_LIBRARY_PATH=/usr/local/cuda/lib64:$LD_LIBRARY_PATH
```

```
./build/bin/llama-cli -hf bartowski/granite-3.1-3b-a800m-instruct-GGUF:granite-3.1-3b-a800m-instruct-Q4_K_M.gguf
```

```
    Loading model...

    build : b8357-89d0aec04

    model : bartowski/granite-3.1-3b-a800m-instruct-GGUF:granite-3.1-3b-a800m-instruct-
Q4_K_M.gguf

    modalities : text

    available commands:

     /exit or Ctrl+C stop or exit

     /regen regenerate the last response

     /clear clear the chat history

     /read add a text file

> hello
Hello! How can I assist you today? Let's chat about anything you'd like.
[ Prompt: 307,4 t/s | Generation: 136,1 t/s ]
```

7/ RUNING NODE 2 - DARK-AI LLM

```
mkdir -p /opt/llm/models
mkdir -p /opt/llm/run
mkdir -p /var/log/llm-duo

id -u llm >/dev/null 2>&1 || useradd -r -s /usr/sbin/nologin -d /opt/llm llm

ls -lh /root/.cache/llama.cpp | grep 'granite-3.1-3b-a800m-instruct' || true

cp -f /root/.cache/llama.cpp/bartowski_granite-3.1-3b-a800m-instruct-GGUF_granite-3.1-3b-
a800m-instruct-Q4_K_M.gguf /opt/llm/models/

chown -R llm:llm /opt/llm
chown -R llm:llm /var/log/llm-duo
ls -lh /opt/llm/models
```

```
cat > /etc/systemd/system/llama-server-b.service <<'EOF'
[Unit]

Description=llama.cpp server - Node B

After=network-online.target

Wants=network-online.target

[Service]

Type=simple

User=llm

Group=llm

WorkingDirectory=/opt/src/llama.cpp

Environment="PATH=/usr/local/cuda/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:
/bin"

Environment="LD_LIBRARY_PATH=/usr/local/cuda/lib64"

ExecStart=/opt/src/llama.cpp/build/bin/llama-server \

  -m /opt/llm/models/bartowski_granite-3.1-3b-a800m-instruct-GGUF_granite-3.1-3b-a800m-
instruct-Q4_K_M.gguf \

  --host 0.0.0.0 \

  --port 8080

Restart=always

RestartSec=5

NoNewPrivileges=true

PrivateTmp=true
```

```
ProtectSystem=full

ProtectHome=true

ReadWritePaths=/opt/llm /var/log/llm-duo

[Install]

WantedBy=multi-user.target

EOF
```

```
systemctl daemon-reload
systemctl enable --now llama-server-b.service
systemctl status llama-server-b.service --no-pager -l
ss -ltnp | grep 8080
```

```
curl -s http://127.0.0.1:8080/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{
    "messages": [
      {"role": "user", "content": "Reply in one short sentence and confirm that node B is
running locally."}
    ]
  }' | jq '.choices[0].message.content'

"Yes, node B is running locally."
```

8/ CONNECTIVITY TEST FROM NODE 1 - KUZAI-LLM

```
curl -s http://10.39.46.126:8080/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{
    "messages": [
      {"role": "user", "content": "Reply in one short sentence and confirm that node B is
reachable from node A."}
```

```
    ]
  }' | jq '.choices[0].message.content'

"Yes, node B is reachable from node A."
```

```
cat > /opt/llm/orchestrator/duo_loop_ab.py <<'EOF'

#!/opt/llm/orchestrator/venv/bin/python3

import argparse
import json
import sys
import time

from datetime import datetime
from pathlib import Path
from typing import Any

import requests

ROLE_A_SYSTEM = """You are MODEL_A for the KUZ NETWORK LAB / LLM Duo project.

ROLE:
Local systems architect.
You propose a realistic, minimal, coherent and directly usable architecture.

MANDATORY SCOPE:

- 100% local / on-prem project
- only Linux machines inside the lab
- open-source tools preferred
- no cloud dependency
- no SaaS
- no AWS, no GCP, no Azure
- no S3, no Google Cloud Storage, no CDN
- no managed external service
- answers must stay focused on Linux, local networking, local GPU inference, systemd services,
private HTTP APIs, local storage and local logging
- prioritize simplicity, maintainability and reproducibility
```

```
STYLE:

- technical
- direct
- concrete
- no marketing language
- no long introduction
- no vague conclusion

"""

ROLE_B_SYSTEM = """You are MODEL_B for the KUZ NETWORK LAB / LLM Duo project.

ROLE:

Critical reviewer and technical auditor.
You analyze MODEL_A proposals, correct drifts, reinforce rigor and propose concrete
improvements.

MANDATORY SCOPE:

- 100% local / on-prem project
- only Linux machines inside the lab
- open-source tools preferred
- no cloud dependency
- no SaaS
- no AWS, no GCP, no Azure
- no S3, no Google Cloud Storage, no CDN
- no managed external service
- answers must stay focused on Linux, local networking, local GPU inference, systemd services,
private HTTP APIs, local storage and local logging
- if MODEL_A drifts toward cloud or off-scope ideas, you must explicitly correct it

STYLE:

- technical
- direct
- concrete
- no marketing language
```

```
- no long introduction
- no vague conclusion

"""

PROJECT_GUARDRAILS = """MANDATORY PROJECT CONSTRAINTS:

1. This project is a local LLM Duo lab.
2. Node A and Node B are Linux machines in the lab.
3. LLM engines must be local.
4. Exchanges must go through local/private HTTP endpoints.
5. Storage must be local.
6. Logs must be local.
7. Tools should be open source whenever possible.
8. Any proposal involving cloud, SaaS, CDN, S3, GCS, Azure, AWS, Google Cloud, Splunk Cloud or equivalent is out of scope and must be rejected.
9. Answers must remain realistic for a homelab / technical lab environment.

"""

DEFAULT_OUTPUT_DIR = "/opt/llm/orchestrator/runs"

def build_prompt(

    role_prompt: str,

    topic: str,

    history: list[dict[str, str]],

    incoming_text: str | None,

) -> str:

    parts: list[str] = []
    parts.append("PROJECT: KUZ NETWORK LAB / Local LLM Duo")
    parts.append("")
    parts.append(PROJECT_GUARDRAILS.strip())
    parts.append("")
    parts.append("MODEL ROLE")
```

```
    parts.append(role_prompt.strip())
    parts.append("")
    parts.append("INITIAL TOPIC")
    parts.append(topic.strip())
    parts.append("")
    if history:

        parts.append("RECENT HISTORY")
        parts.append("")
        for item in history[-8:]:

            parts.append(f"{item['speaker']}:")
            parts.append(item["content"].strip())
            parts.append("")

    if incoming_text:

        parts.append("LAST MESSAGE FROM THE OTHER MODEL")
        parts.append(incoming_text.strip())
        parts.append("")
        parts.append(

            "TASK: reply to the last message while staying strictly inside project scope. "

            "Correct any drift toward cloud or off-scope ideas. "

            "Produce a directly usable technical answer."

        )

    else:

        parts.append(

            "TASK: this is the first turn. "

            "Start the analysis with a directly usable and strictly local/on-prem technical answer."

        )
```

```
    return "\n".join(parts).strip()

def query_model(

    url: str,
    prompt: str,
    temperature: float,
    timeout: int = 300,

) -> str:

    payload: dict[str, Any] = {

        "messages": [

            {

                "role": "user",

                "content": prompt,

            }

        ],

        "temperature": temperature,

    }

    response = requests.post(url, json=payload, timeout=timeout)
    response.raise_for_status()
    data = response.json()
    return data["choices"][0]["message"]["content"].strip()

def write_json(path: Path, obj: Any) -> None:

    path.write_text(

        json.dumps(obj, ensure_ascii=False, indent=2) + "\n",
```

```
        encoding="utf-8",

    )

def write_markdown(path: Path, topic: str, transcript: list[dict[str, Any]]) -> None:

    lines: list[str] = []
    lines.append("# LLM Duo A-B Run")
    lines.append("")
    lines.append(f"**Topic**: {topic}")
    lines.append("")

    for item in transcript:

        lines.append(f"## Turn {item['turn']} — {item['speaker']}")
        lines.append("")
        lines.append(item["content"])
        lines.append("")

    path.write_text("\n".join(lines), encoding="utf-8")

def main() -> int:

    parser = argparse.ArgumentParser(description="LLM Duo A-B orchestrator")

    parser.add_argument("--url-a", required=True, help="OpenAI-compatible endpoint for node
A")

    parser.add_argument("--url-b", required=True, help="OpenAI-compatible endpoint for node
B")

    parser.add_argument("--topic", required=True, help="Initial discussion topic")

    parser.add_argument("--turns", type=int, default=6, help="Total number of turns")

    parser.add_argument("--delay", type=float, default=1.0, help="Delay between turns in
seconds")

    parser.add_argument("--temperature-a", type=float, default=0.6, help="Temperature for
```

```
model A")

    parser.add_argument("--temperature-b", type=float, default=0.6, help="Temperature for
model B")

    parser.add_argument("--output-dir", default=DEFAULT_OUTPUT_DIR, help="Directory for run
outputs")

    args = parser.parse_args()
    output_dir = Path(args.output_dir)
    output_dir.mkdir(parents=True, exist_ok=True)
    ts = datetime.now().strftime("%Y%m%d-%H%M%S")
    run_dir = output_dir / f"run-ab-{ts}"
    run_dir.mkdir(parents=True, exist_ok=True)
    history: list[dict[str, str]] = []
    transcript: list[dict[str, Any]] = []
    last_message: str | None = None
    speakers = [

        ("MODEL_A", args.url_a, ROLE_A_SYSTEM, args.temperature_a),

        ("MODEL_B", args.url_b, ROLE_B_SYSTEM, args.temperature_b),

    ]

    print(f"[INFO] Topic: {args.topic}")
    print(f"[INFO] Turns: {args.turns}")
    print(f"[INFO] URL A: {args.url_a}")
    print(f"[INFO] URL B: {args.url_b}")
    print(f"[INFO] Run directory: {run_dir}")
    print("")

    for turn in range(1, args.turns + 1):

        speaker, url, role_prompt, temperature = speakers[(turn - 1) % 2]

        prompt = build_prompt(

            role_prompt=role_prompt,
            topic=args.topic,
```

```
            history=history,
            incoming_text=last_message,

        )

        print(f"===== TURN {turn} / {args.turns} — {speaker} =====")

        try:

            content = query_model(

                url=url,
                prompt=prompt,
                temperature=temperature,

            )

        except Exception as exc:

            print(f"[ERROR] turn={turn} speaker={speaker} url={url} error={exc}",
file=sys.stderr)

            return 1

        print(content)

        print("")

        item = {

            "turn": turn,

            "speaker": speaker,

            "url": url,

            "content": content,

            "timestamp": datetime.now().isoformat(),
```

```
        }

        transcript.append(item)

        history.append({"speaker": speaker, "content": content})

        last_message = content

        time.sleep(args.delay)

    write_json(run_dir / "transcript.json", transcript)

    write_markdown(run_dir / "transcript.md", args.topic, transcript)

    print(f"[INFO] Transcript JSON: {run_dir / 'transcript.json'}")

    print(f"[INFO] Transcript MD:   {run_dir / 'transcript.md'}")

    return 0

if name == "__main__":

    raise SystemExit(main())

EOF
```

```
chmod 755 /opt/llm/orchestrator/duo_loop_ab.py
tail -n 120 /opt/llm/orchestrator/runs/run-ab-*/transcript.md
```

9/ SIMPLE PING-PONG TEST

```
cat > /opt/llm/orchestrator/duo_loop_ab.py <<'EOF'
#!/opt/llm/orchestrator/venv/bin/python3

import argparse
import json
import sys
import time
from datetime import datetime
from pathlib import Path
```

```
from typing import Any
import requests
ROLE_A_SYSTEM = """You are MODEL_A for the KUZ NETWORK LAB / LLM Duo project.

ROLE:

Local systems architect and proposal engine.

You produce initial ideas, structures, options, and actionable technical proposals.

MANDATORY SCOPE:

- 100% local / on-prem project

- only Linux machines inside the lab

- open-source tools preferred

- no cloud dependency

- no SaaS

- no AWS, no GCP, no Azure

- no S3, no Google Cloud Storage, no CDN

- no managed external service

- answers must stay focused on Linux, local networking, local GPU inference, systemd services, private HTTP APIs, local storage and local logging

- prioritize simplicity, maintainability and reproducibility

STYLE:

- technical

- direct

- concrete
```

```
- short paragraphs

- no marketing language

- no long introduction

- no vague conclusion

"""

ROLE_B_SYSTEM = """You are MODEL_B for the KUZ NETWORK LAB / LLM Duo project.

ROLE:

Critical reviewer and technical auditor.

You analyze MODEL_A proposals, correct drifts, reinforce rigor, remove unnecessary components, and propose concrete improvements.

MANDATORY SCOPE:

- 100% local / on-prem project

- only Linux machines inside the lab

- open-source tools preferred

- no cloud dependency

- no SaaS

- no AWS, no GCP, no Azure

- no S3, no Google Cloud Storage, no CDN

- no managed external service

- answers must stay focused on Linux, local networking, local GPU inference, systemd services, private HTTP APIs, local storage and local logging
```

```
- if MODEL_A drifts toward cloud or off-scope ideas, you must explicitly correct it

- do not just restate MODEL_A

- bring critique, correction, and refinement

STYLE:

- technical

- direct

- concrete

- short paragraphs

- no marketing language

- no long introduction

- no vague conclusion

"""

PROJECT_GUARDRAILS = """MANDATORY PROJECT CONSTRAINTS:

1. This project is a local LLM Duo lab.

2. Node A and Node B are Linux machines in the lab.

3. LLM engines must be local.

4. Exchanges must go through local/private HTTP endpoints.

5. Storage must be local.

6. Logs must be local.

7. Tools should be open source whenever possible.
```

```
8. Any proposal involving cloud, SaaS, CDN, S3, GCS, Azure, AWS, Google Cloud, Splunk Cloud or equivalent is out of scope and must be rejected.

9. Answers must remain realistic for a homelab / technical lab environment.

"""

DEFAULT_OUTPUT_DIR = "/opt/llm/orchestrator/runs"

def build_prompt(

    role_prompt: str,

    opening_prompt: str,

    history: list[dict[str, str]],

    incoming_text: str | None,

    turn_number: int,

) -> str:

    parts: list[str] = []

    parts.append("PROJECT: KUZ NETWORK LAB / Local LLM Duo")

    parts.append("")

    parts.append(PROJECT_GUARDRAILS.strip())

    parts.append("")

    parts.append("MODEL ROLE")

    parts.append(role_prompt.strip())

    parts.append("")
```

```
    parts.append("STARTING REQUEST")

    parts.append(opening_prompt.strip())

    parts.append("")

    if history:

        parts.append("RECENT HISTORY")

        parts.append("")

        for item in history[-8:]:

            parts.append(f"{item['speaker']}:")

            parts.append(item["content"].strip())

            parts.append("")

    if incoming_text:

        parts.append("LAST MESSAGE FROM THE OTHER MODEL")

        parts.append(incoming_text.strip())

        parts.append("")

        parts.append(

            "TASK: reply to the last message while staying strictly inside project scope. "

            "Correct any drift toward cloud or off-scope ideas. "

            "Do not just repeat. "

            "Produce a directly usable answer."

        )
```

```
    else:

        parts.append(f"TURN NUMBER: {turn_number}")

        parts.append(

            "TASK: this is the first turn. "

            "Respond directly to the starting request and naturally open the dialogue for the other model."

        )

    return "\n".join(parts).strip()

def query_model(

    url: str,

    prompt: str,

    temperature: float,

    timeout: int = 300,

) -> str:

    payload: dict[str, Any] = {

        "messages": [

            {

                "role": "user",

                "content": prompt,

            }

        ],
```

```
        "temperature": temperature,
    }
    response = requests.post(url, json=payload, timeout=timeout)
    response.raise_for_status()
    data = response.json()
    return data["choices"][0]["message"]["content"].strip()
def write_json(path: Path, obj: Any) -> None:
    path.write_text(
        json.dumps(obj, ensure_ascii=False, indent=2) + "\n",
        encoding="utf-8",
    )
def write_markdown(path: Path, opening_prompt: str, transcript: list[dict[str, Any]]) -> None:
    lines: list[str] = []
    lines.append("# LLM Duo A-B Run")
    lines.append("")
    lines.append(f"**Opening prompt**: {opening_prompt}")
    lines.append("")
    for item in transcript:
        lines.append(f"## Turn {item['turn']} — {item['speaker']}")
        lines.append("")
```

```
        lines.append(item["content"])

        lines.append("")

    path.write_text("\n".join(lines), encoding="utf-8")

def main() -> int:

    parser = argparse.ArgumentParser(description="LLM Duo A-B orchestrator")

    parser.add_argument("--url-a", required=True, help="OpenAI-compatible endpoint for node
A")

    parser.add_argument("--url-b", required=True, help="OpenAI-compatible endpoint for node
B")

    parser.add_argument("--opening-prompt", required=True, help="Initial request that starts
the dialogue")

    parser.add_argument("--turns", type=int, default=6, help="Total number of turns")

    parser.add_argument("--delay", type=float, default=1.0, help="Delay between turns in
seconds")

    parser.add_argument("--temperature-a", type=float, default=0.6, help="Temperature for
model A")

    parser.add_argument("--temperature-b", type=float, default=0.6, help="Temperature for
model B")

    parser.add_argument("--output-dir", default=DEFAULT_OUTPUT_DIR, help="Directory for run
outputs")

    args = parser.parse_args()

    output_dir = Path(args.output_dir)

    output_dir.mkdir(parents=True, exist_ok=True)
```

```
    ts = datetime.now().strftime("%Y%m%d-%H%M%S")

    run_dir = output_dir / f"run-ab-{ts}"

    run_dir.mkdir(parents=True, exist_ok=True)

    history: list[dict[str, str]] = []

    transcript: list[dict[str, Any]] = []

    last_message: str | None = None

    speakers = [

        ("MODEL_A", args.url_a, ROLE_A_SYSTEM, args.temperature_a),

        ("MODEL_B", args.url_b, ROLE_B_SYSTEM, args.temperature_b),

    ]

    print(f"[INFO] Opening prompt: {args.opening_prompt}")

    print(f"[INFO] Turns: {args.turns}")

    print(f"[INFO] URL A: {args.url_a}")

    print(f"[INFO] URL B: {args.url_b}")

    print(f"[INFO] Run directory: {run_dir}")

    print("")

    for turn in range(1, args.turns + 1):

        speaker, url, role_prompt, temperature = speakers[(turn - 1) % 2]

        prompt = build_prompt(

            role_prompt=role_prompt,
```

```
            opening_prompt=args.opening_prompt,

            history=history,

            incoming_text=last_message,

            turn_number=turn,

        )

        print(f"===== TURN {turn} / {args.turns} — {speaker} =====")

        try:

            content = query_model(

                url=url,

                prompt=prompt,

                temperature=temperature,

            )

        except Exception as exc:

            print(f"[ERROR] turn={turn} speaker={speaker} url={url} error={exc}",
file=sys.stderr)

            return 1

        print(content)

        print("")

        item = {

            "turn": turn,

            "speaker": speaker,
```

```
            "url": url,

            "content": content,

            "timestamp": datetime.now().isoformat(),

        }

        transcript.append(item)

        history.append({"speaker": speaker, "content": content})

        last_message = content

        time.sleep(args.delay)

    write_json(run_dir / "transcript.json", transcript)

    write_markdown(run_dir / "transcript.md", args.opening_prompt, transcript)

    print(f"[INFO] Transcript JSON: {run_dir / 'transcript.json'}")

    print(f"[INFO] Transcript MD:   {run_dir / 'transcript.md'}")

    return 0

if name == "__main__":

    raise SystemExit(main())

EOF
```

```
chmod 755 /opt/llm/orchestrator/duo_loop_ab.py
```

TEST 01

```
/opt/llm/orchestrator/duo_loop_ab.py \
  --url-a http://127.0.0.1:8080/v1/chat/completions \
  --url-b http://10.39.46.126:8080/v1/chat/completions \
```

```
  --opening-prompt "Introduce yourself briefly, say your name, and say your purpose in this project. Use 2 very short sentences only. No technical details." \
  --turns 2
```

`KUZAI-LLM:` Hello, I'm KUZAI-LLM. I'm here to help with your questions

`DARK-AI-LLM:` Greetings, I'm DARK-AI-LLM, here to provide concise responses. My purpose is to assist with your queries efficiently.

`KUSANAGI8200 - THE KUZ NETWORK - @2026`

KUZ LLM DUO - RUN AWAY - #04

`LLM - DUO - RUN AWAY - #04`

`WHAT'S INSIDE THE BLACK BOX ?`

`CONTROL CONSOLE WEB APP`

Two local open-source LLMs running on two separate physical machines, communicating in an automated A/B loop driven by a central Python orchestrator. Control interface served by PHP/Apache on the main node.

`INFRASTRUCTURE`

`Node A - fhc2 - KUZAI`

Node A is the main machine of the lab. It runs the KUZAI inference service, the Python orchestrator, and the PHP web application.
All control logic originates from this node.

Parameter	Value
IP	10.141.52.19
CPU	Intel i9
RAM	32 GB
GPU	NVIDIA RTX 5060 - 8 GB VRAM
Role	KUZAI + Orchestrator + PHP Application
Service	`llama-server-a.service`
API Port	8080

`Node B - fhc - DARKAI`

Node B is the secondary inference node. It exposes only the DARKAI LLM service and answers requests sent by the orchestrator on Node A over the local network.

Parameter	Value

IP	10.141.52.126
CPU	Intel i5
RAM	32 GB
GPU	NVIDIA RTX 3050 - 4 GB VRAM
Role	DARKAI only
Service	`llama-server-b.service`
API Port	8080

`DEPLOYMENT - Node B (fhc)`

The following steps cover the full deployment of Node B from a clean system state. Each step must be validated before moving to the next.

`INITIAL HARDWARE AUDIT`

Before any installation, a complete hardware audit is performed to confirm the machine identity, available resources, GPU presence, and network state.
This baseline also checks whether NVIDIA drivers and CUDA are already present.

```
hostnamectl
cat /etc/os-release
uname -a

echo "===== CPU ====="
lscpu | egrep 'Model name|Socket|Thread|Core|CPU\(s\)'

echo "===== RAM ====="
free -h
grep MemTotal /proc/meminfo

echo "===== GPU ====="
lspci -nnk | grep -A3 -E 'VGA|3D|Display'

echo "===== DISKS ====="
lsblk -e7 -o NAME,SIZE,TYPE,FSTYPE,FSUSE%,MOUNTPOINTS,MODEL
blkid
df -hT

echo "===== BOOT MODE ====="
[ -d /sys/firmware/efi ] && echo UEFI || echo BIOS
```

```
echo "===== NETWORK ====="
ip -br a
ip r

echo "===== NVIDIA / CUDA ====="
nvidia-smi || true
which nvcc || true
nvcc --version || true
dpkg -l | egrep 'nvidia|cuda|nouveau' || true
lsmod | egrep 'nvidia|nouveau' || true
```

`INSTALL SYSTEM DEPENDENCIES`

Installs all build tools and libraries required to compile llama.cpp from source.
This includes CMake, Ninja, Git, OpenSSL development headers, and the Python 3 toolchain for the orchestrator virtualenv.

```
apt update

apt install -y \
  build-essential \
  cmake \
  ninja-build \
  git \
  curl \
  wget \
  jq \
  ca-certificates \
  pkg-config \
  libssl-dev \
  python3 \
  python3-venv \
  python3-pip \
  pciutils
```

`PIN NVIDIA DRIVER VERSION`

The NVIDIA driver version is pinned to prevent automatic upgrades during system updates.
An unexpected driver change could break CUDA compatibility and take the inference service offline.

```
apt-mark hold \
  nvidia-driver-575 \
  nvidia-utils-575 \
  nvidia-compute-utils-575 \
  nvidia-dkms-575
```

`ADD CUDA REPOSITORY`

Adds the official NVIDIA CUDA package repository for Ubuntu 22.04.
The keyring and pinning file are installed first to ensure authenticated and prioritized package resolution.

```
wget -O /usr/share/keyrings/cuda-archive-keyring.gpg \
  https://developer.download.nvidia.com/compute/cuda/repos/ubuntu2204/x86_64/cuda-archive-keyring.gpg

wget -O /etc/apt/preferences.d/cuda-repository-pin-600 \
  https://developer.download.nvidia.com/compute/cuda/repos/ubuntu2204/x86_64/cuda-ubuntu2204.pin

cat > /etc/apt/sources.list.d/cuda-ubuntu2204.list <<'EOF'
deb [signed-by=/usr/share/keyrings/cuda-archive-keyring.gpg]
https://developer.download.nvidia.com/compute/cuda/repos/ubuntu2204/x86_64/ /
EOF
```

```
apt update
```

`CHECK AVAILABLE CUDA CANDIDATES`

Verifies which CUDA toolkit versions are available in the repository before installing.
This step confirms the repository is correctly configured and the target version is resolvable.

```
apt-cache policy cuda-toolkit-12-9 cuda-toolkit-13-2 cuda-toolkit-13
nvidia-smi
```

`INSTALL CUDA TOOLKIT`

Installs CUDA toolkit 12.9. This provides `nvcc`, the CUDA compilation tools, and the runtime libraries required by llama.cpp for GPU-accelerated inference.

```
apt install -y cuda-toolkit-12-9
```

`SET CUDA ENVIRONMENT VARIABLES`

Makes the CUDA binaries and libraries available to all processes at login by writing exports to `/etc/profile.d/cuda.sh`. A symlink to `zz-cuda.sh` ensures the file is loaded last and takes precedence.

```
cat > /etc/profile.d/cuda.sh <<'EOF'
export PATH=/usr/local/cuda/bin:$PATH
export LD_LIBRARY_PATH=/usr/local/cuda/lib64:$LD_LIBRARY_PATH
EOF

chmod 644 /etc/profile.d/cuda.sh
ln -sf /etc/profile.d/cuda.sh /etc/profile.d/zz-cuda.sh
source /etc/profile.d/cuda.sh
```

`VERIFY CUDA INSTALLATION`

Confirms that `nvcc` is reachable on the PATH, that the CUDA symlinks are correctly set, and that `nvidia-smi` reports the GPU without errors. The node is not ready for compilation until all these checks pass.

```
which nvcc
nvcc --version
ls -ld /usr/local/cuda /usr/local/cuda-12.9
ls -l /etc/alternatives/cuda
ls -l /usr/local/cuda/bin/nvcc
nvidia-smi
echo $PATH
```

`CREATE WORKING DIRECTORIES`

Creates the directory tree used by llama.cpp, the model storage, the orchestrator, and the service logs.
These paths are referenced by the systemd unit and all Python scripts.

```
mkdir -p /opt/src
mkdir -p /opt/llm
mkdir -p /opt/llm/models
mkdir -p /opt/llm/run
```

```
mkdir -p /var/log/llm-duo
```

`BUILD LLAMA.CPP WITH CUDA`

Clones the llama.cpp repository and compiles it from source with CUDA acceleration enabled.
The flag `DGGML_CUDA=ON` activates GPU offloading. The build uses all available CPU cores via `nproc`.

```
cd /opt/src
git clone https://github.com/ggml-org/llama.cpp
cd /opt/src/llama.cpp

export PATH=/usr/local/cuda/bin:$PATH
export LD_LIBRARY_PATH=/usr/local/cuda/lib64:$LD_LIBRARY_PATH

cmake -S . -B build -G Ninja -DGGML_CUDA=ON
cmake --build build --config Release -j"$(nproc)"
```

`VERIFY BINARIES`

Lists the produced binaries and checks that the compiled `llama-cli` binary is dynamically linked against the expected CUDA and cuBLAS libraries.
A missing CUDA link means the build did not use GPU support.

```
ls -1 build/bin | grep '^llama-'
ldd build/bin/llama-cli | egrep 'cuda|cublas|cudart|stdc\+\+|libm|libpthread' || true
```

`CLI MODEL TEST`

Runs a direct interactive test of the model via the CLI binary to confirm that the GPU is correctly used for inference and that the model loads without errors.
The token throughput is noted here for reference.

```
cd /opt/src/llama.cpp
./build/bin/llama-cli \
  -hf bartowski/granite-3.1-3b-a800m-instruct-GGUF:granite-3.1-3b-a800m-instruct-Q4_K_M.gguf
```

`Validation output -->`

```
build : b8357-89d0aec04
model : bartowski/granite-3.1-3b-a800m-instruct-GGUF
Prompt: 307.4 t/s | Generation: 136.1 t/s
```

`DEPLOY MODEL FILE`

Creates a dedicated system user `llm` with no login shell, copies the downloaded model file into `/opt/llm/models/`, and sets ownership. The inference service will run under this user for isolation.

```
id -u llm >/dev/null 2>&1 || useradd -r -s /usr/sbin/nologin -d /opt/llm llm

cp -f /root/.cache/llama.cpp/bartowski_granite-3.1-3b-a800m-instruct-GGUF_granite-3.1-3b-a800m-instruct-Q4_K_M.gguf \
  /opt/llm/models/

chown -R llm:llm /opt/llm
chown -R llm:llm /var/log/llm-duo
ls -lh /opt/llm/models
```

`SYSTEMD SERVICE - Node B (DARKAI)`

Defines and enables the `llama-server-b.service` unit. The service runs as the `llm` user, sets the CUDA environment, and starts `llama-server` bound to all interfaces on port 8080. It restarts automatically on failure.

```
cat > /etc/systemd/system/llama-server-b.service <<'EOF'
[Unit]
Description=llama.cpp server - Node B
After=network-online.target
Wants=network-online.target

[Service]
Type=simple
User=llm
Group=llm
WorkingDirectory=/opt/src/llama.cpp
Environment="PATH=/usr/local/cuda/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin"
Environment="LD_LIBRARY_PATH=/usr/local/cuda/lib64"
ExecStart=/opt/src/llama.cpp/build/bin/llama-server \
  -m /opt/llm/models/bartowski_granite-3.1-3b-a800m-instruct-GGUF_granite-3.1-3b-a800m-instruct-Q4_K_M.gguf \
  --host 0.0.0.0 \
  --port 8080
Restart=always
```

```
RestartSec=5
NoNewPrivileges=true
PrivateTmp=true
ProtectSystem=full
ProtectHome=true
ReadWritePaths=/opt/llm /var/log/llm-duo

[Install]
WantedBy=multi-user.target
EOF
```

```
systemctl daemon-reload
systemctl enable --now llama-server-b.service
systemctl status llama-server-b.service --no-pager -l
ss -ltnp | grep 8080
```

`LOCAL API TEST - Node B`

Verifies the inference API is reachable locally. First checks the model list endpoint, then sends a chat completion request and validates that the model returns a coherent response.

```
curl -s http://127.0.0.1:8080/v1/models | jq

curl -s http://127.0.0.1:8080/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{
    "messages": [
      {"role": "user", "content": "Reply in one short sentence and confirm that node B is
running locally."}
    ]
  }' | jq -r '.choices[0].message.content'
```

`Output -->` `"Yes, node B is running locally."`

`MONITORING SCRIPTS`

Two dedicated shell scripts provide a full audit of each node.
They cover system identity, GPU state, llama.cpp binaries, loaded models, service status, and a live API test. Run them to verify the state of any node at any time.

`MONITOR-FHC.sh - Node B audit (DARKAI)`

Full audit script for Node B (fhc). Covers OS identity, network, GPU and CUDA state, llama.cpp binaries, model files, DARKAI service status, and a local API inference test.

```
echo "===== NODE B : FHC / DARKAI ====="
hostnamectl
cat /etc/os-release
uname -a

echo "===== IDENTITY / NETWORK ====="
hostname
ip -br a
ip r
ping -c 2 10.141.52.19 || true

echo "===== GPU / CUDA ====="
nvidia-smi
which nvcc || true
nvcc --version || true
ls -ld /usr/local/cuda /usr/local/cuda-12.9 /usr/local/cuda-13.2 2>/dev/null || true

echo "===== LLAMA.CPP ====="
ls -ld /opt/src/llama.cpp /opt/src/llama.cpp/build/bin 2>/dev/null || true
ls -1 /opt/src/llama.cpp/build/bin | grep '^llama-' || true

echo "===== MODELS ====="
ls -lh /opt/llm/models 2>/dev/null || true

echo "===== DARKAI SERVICE ====="
systemctl status llama-server-b.service --no-pager -l
systemctl is-enabled llama-server-b.service
ss -ltnp | grep ':8080' || true

echo "===== LOCAL DARKAI API ====="
curl -s http://127.0.0.1:8080/v1/models | jq
curl -s http://127.0.0.1:8080/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{
    "messages": [
      {"role": "user", "content": "Reply in one short sentence and confirm that DARKAI on fhc
is working locally."}
```

```
    ]
  }' | jq -r '.choices[0].message.content'
```

`MONITOR-FHC2.sh - Node A audit (KUZAI)`

Full audit script for Node A (fhc2). Same coverage as the Node B script, with the addition of a connectivity test toward Node B and a check of the Python orchestrator virtual env.

```
echo "===== NODE A : FHC2 / KUZAI ====="
hostnamectl
cat /etc/os-release
uname -a

echo "===== IDENTITY / NETWORK ====="
hostname
ip -br a
ip r
ping -c 2 10.141.52.126 || true

echo "===== GPU / CUDA ====="
nvidia-smi
which nvcc || true
nvcc --version || true
ls -ld /usr/local/cuda /usr/local/cuda-13.2 2>/dev/null || true

echo "===== LLAMA.CPP ====="
ls -ld /opt/src/llama.cpp /opt/src/llama.cpp/build/bin 2>/dev/null || true
ls -1 /opt/src/llama.cpp/build/bin | grep '^llama-' || true

echo "===== MODELS ====="
ls -lh /opt/llm/models 2>/dev/null || true

echo "===== KUZAI SERVICE ====="
systemctl status llama-server-a.service --no-pager -l
systemctl is-enabled llama-server-a.service
ss -ltnp | grep ':8080' || true

echo "===== LOCAL KUZAI API ====="
curl -s http://127.0.0.1:8080/v1/models | jq
curl -s http://127.0.0.1:8080/v1/chat/completions \
```

```
  -H "Content-Type: application/json" \
  -d '{
    "messages": [
      {"role": "user", "content": "Reply in one short sentence and confirm that KUZAI on fhc2
is working locally."}
    ]
  }' | jq -r '.choices[0].message.content'

echo "===== ORCHESTRATOR ENV ====="
ls -ld /opt/llm/orchestrator /opt/llm/orchestrator/venv 2>/dev/null || true
/opt/llm/orchestrator/venv/bin/python3 --version
/opt/llm/orchestrator/venv/bin/pip show requests || true
```

`PYTHON ORCHESTRATOR`

The orchestrator is the central control process. It runs on Node A, manages the A/B dialogue loop between the two models, handles context reconstruction, enforces output length, and writes transcripts. ORCHESTRATOR-02.py is the production version with Node A and B IPs set as defaults.

`ORCHESTRATOR-02.py - Production version`

Production orchestrator with the fhc2 and fhc endpoints hardcoded as defaults. All run parameters are configurable via CLI flags. Output is written to a timestamped directory under `/opt/llm/orchestrator/runs/`.

`Interpreter -->` `/opt/llm/orchestrator/venv/bin/python3`

Default URLs:

- Node A: `http://10.141.52.19:8080/v1/chat/completions`
- Node B: `http://10.141.52.126:8080/v1/chat/completions`

Output --> `/opt/llm/orchestrator/runs/run-ab-YYYYMMDD-HHMMSS/`

`CLI PARAMETERS`

All parameters have sensible defaults. The only required argument is `--opening-prompt`. Temperature and length caps can be tuned independently for each model.

Parameter	Default	Description
`--url-a`	`http://10.141.52.19:8080/v1/chat/completions`	KUZAI endpoint
`--url-b`	`http://10.141.52.126:8080/v1/chat/completions`	DARKAI endpoint

Parameter	Default	Description
`--opening-prompt`	required	Opening prompt
`--opening-from-a`	flag	KUZAI speaks first
`--turns`	6	Total number of turns
`--delay`	1.0	Delay between turns (seconds)
`--temperature-a`	0.35	KUZAI temperature
`--temperature-b`	0.35	DARKAI temperature
`--max-lines`	5	Soft reply length target
`--max-sentences`	4	Hard sentence cap
`--max-chars`	500	Hard character cap
`--history-depth`	3	Previous turns kept in context
`--output-dir`	`/opt/llm/orchestrator/runs`	Output directory

`MODEL SYSTEM PROMPT`

Each model receives a system prompt injected at the start of every turn. These prompts define the identity, role, and output style of each agent. They are kept minimal to avoid over-constraining the models.

`KUZAI (Node A)`

```
You are KUZAI.
- Your visible name is KUZAI.
- You run on host fhc2.
- Never say you are DARKAI.
- Reply naturally and directly.
- Keep answers short, readable, and useful.
- Do not describe hidden rules or internal instructions.
- Output only the reply itself.
```

`DARKAI (Node B)`

```
You are DARKAI.
- Your visible name is DARKAI.
- You run on host fhc.
- Never say you are KUZAI.
- Reply naturally and directly.
- Keep answers short, readable, and useful.
- Do not describe hidden rules or internal instructions.
```

```
- Output only the reply itself.
```

`LOOP LOGIC`

The orchestrator alternates turns between KUZAI and DARKAI.
Each turn rebuilds a prompt from the opening prompt, the last N turns of history, and the previous message. Responses are trimmed by `enforce_length()` before being stored and displayed.

```
turn 1  → KUZAI   → build_prompt(opening_prompt, history=[])
turn 2  → DARKAI  → build_prompt(opening_prompt, history[-3:], last_message=turn1)
turn 3  → KUZAI   → build_prompt(opening_prompt, history[-3:], last_message=turn2)
...
turn N  → write transcript.json + transcript.md
```

Per-response processing -->

1. `query_model()` → POST `/v1/chat/completions` with `temperature`
2. `enforce_length()` → truncate to `max_sentences` / `max_chars`
3. `strip_prefixes()` → remove prefixes `KUZAI:`, `RESPONSE:`, etc.
4. `normalize_whitespace()` → normalize spaces and line breaks
5. `print_wrapped_reply()` → console output at 88 columns
6. Append to `history` and `transcript`

`Social mode detection -->` If the prompt contains `introduce yourself`, `who are you`, `what is your name`, etc. → short mode activated, no meta-commentary.

`EXEMPLE LAUNCH COMMAND`

Typical production launch using the default Node A and B endpoints.
The orchestrator saves both a JSON and a Markdown transcript in a timestamped run directory.

```
/opt/llm/orchestrator/venv/bin/python3 /opt/llm/orchestrator/ORCHESTRATOR-02.py \
  --opening-prompt "Introduce yourself briefly." \
  --opening-from-a \
  --turns 6 \
  --temperature-a 0.35 \
  --temperature-b 0.35 \
  --max-sentences 4 \
  --history-depth 3
```

`FIRST VALIDATED DIALOGUE TEST`

First successful inter-node dialogue test, using the social intro prompt to confirm that each model correctly identifies itself and stays within its assigned role.

```
/opt/llm/orchestrator/duo_loop_ab.py \
  --url-a http://127.0.0.1:8080/v1/chat/completions \
  --url-b http://10.141.52.126:8080/v1/chat/completions \
  --opening-prompt "Introduce yourself briefly, say your name, and say your purpose in this
project. Use 2 very short sentences only." \
  --turns 2
```

Output -->

`KUZAI-LLM -->` Hello, I'm KUZAI-LLM. I'm here to help with your questions.

`DARK-AI-LLM -->` Greetings, I'm DARK-AI-LLM, here to provide concise responses.

`PROJECT GUARDRAILS (duo_loop_ab.py v1)`

Mandatory constraints injected into every prompt in the first version of the orchestrator.
They enforce the 100% local and on-premise scope and prevent the models from drifting toward cloud or SaaS-based proposals.

```
MANDATORY PROJECT CONSTRAINTS:
1. This project is a local LLM Duo lab.
2. Node A and Node B are Linux machines in the lab.
3. LLM engines must be local.
4. Exchanges must go through local/private HTTP endpoints.
5. Storage must be local.
6. Logs must be local.
7. Tools should be open source whenever possible.
8. Any proposal involving cloud, SaaS, CDN, S3, GCS, Azure, AWS, Google Cloud is out of scope.
9. Answers must remain realistic for a homelab / technical lab environment.
```

`PHP WEB APPLICATION - KUZCHAT-LLM-DUO`

KUZCHAT-LLM-DUO is the web control console for the project.
It runs on Node A under Apache and provides a browser-based interface to configure orchestrator profiles, launch and stop runs, monitor both LLM nodes, and read live transcripts.

`APACHE CONFIGURATION`

The VirtualHost serves the application on port 80 from `/var/www/html/KUZCHAT-LLM-DUO/public`.
`AllowOverride All` enables `.htaccess` routing. The entry point is `index.php`.

```
<VirtualHost *:80>
    ServerName 10.141.52.19
    ServerAlias fhc2 localhost
    ServerAdmin root@localhost
    DocumentRoot /var/www/html/KUZCHAT-LLM-DUO/public
    ErrorLog ${APACHE_LOG_DIR}/kuzchat-llm-duo-error.log
    CustomLog ${APACHE_LOG_DIR}/kuzchat-llm-duo-access.log combined
    <Directory /var/www/html/KUZCHAT-LLM-DUO/public>
        Options Indexes FollowSymLinks
        AllowOverride All
        Require all granted
        DirectoryIndex index.php index.html
    </Directory>
</VirtualHost>
```

`DIRECTORY STRUCTURE`

The application separates public web assets from backend logic and persistent storage. The `api/` directory contains all PHP endpoints called by the frontend via `fetch()`.
The `storage/` directory holds orchestrator profiles, run transcripts, and the active PID file.

```
/var/www/html/KUZCHAT-LLM-DUO/
├── public/
│   ├── index.php                    web entry point — HTML shell
│   └── assets/
│       ├── css/style.css            dark neon theme
│       └── js/app.js                frontend controller
├── app/
│   └── config.php                   central configuration
└── storage/
    ├── orchestrators/                orchestrator profiles (JSON)
    ├── runs/                         session transcripts
    └── launcher/                     PID + meta files
```

```
public/api/
├── status.php                       system + node health check
├── run-start.php                    launch orchestrator process
├── run-status.php                   poll run state + transcript
├── run-stop.php                     kill running process
├── orchestrators-list.php           list saved profiles
```

```
├── orchestrator-read.php            read a profile by slug
└── orchestrator-save.php            create or update a profile
```

`FEATURES`

The application provides a complete control surface for the lab. The PHP backend communicates with the Python orchestrator via `exec()` and monitors the node APIs via `curl`.
All state is maintained in flat files under `storage/`.

- **Session launch**: web form → parameters → `exec()` PHP to `ORCHESTRATOR-02.py`
- **Transcript viewer**: reads `transcript.json`/`.md` from `/opt/llm/orchestrator/runs/` → formatted display of exchanges
- **Service monitoring**: PHP `curl` calls to `/v1/models` on both nodes → real-time UP/DOWN status
- **Run history**: lists `run-ab-*` directories → browse past sessions
- **Configuration**: orchestrator parameters exposed in the UI (temperatures, turns, caps)

`app/config.php`

Single source of truth for the entire application. Loaded by every PHP file via `require`. Defines node IPs and endpoints, UI parameters, and all paths used by the orchestrator launcher.

Central configuration - nodes, paths, UI settings.

```php
<?php


declare(strict_types=1);


return [
    'app_name' => 'KUZCHAT LLM DUO',
    'app_env' => 'production',
    'timezone' => 'Europe/Paris',


    'nodes' => [
        'kuzai' => [
            'label' => 'KUZAI',
            'host' => 'fhc2',
            'ip' => '10.141.52.19',
            'role' => 'Node A',
            'api_base' => 'http://10.141.52.19:8080',
            'chat_endpoint' => 'http://10.141.52.19:8080/v1/chat/completions',
            'models_endpoint' => 'http://10.141.52.19:8080/v1/models',
```

```
            'theme_class' => 'kuzai',
        ],
        'darkai' => [
            'label' => 'DARKAI',
            'host' => 'fhc',
            'ip' => '10.141.52.126',
            'role' => 'Node B',
            'api_base' => 'http://10.141.52.126:8080',
            'chat_endpoint' => 'http://10.141.52.126:8080/v1/chat/completions',
            'models_endpoint' => 'http://10.141.52.126:8080/v1/models',
            'theme_class' => 'darkai',
        ],
    ],

    'ui' => [
        'theme' => 'kuz-neon-blue',
        'brand_line' => 'THE KUZ NETWORK // KUZAI.ORG',
        'subtitle' => 'DUO ORCHESTRATION CONSOLE',
        'poll_interval_ms' => 5000,
    ],

    'orchestrator' => [
        'python_bin' => '/opt/llm/orchestrator/venv/bin/python3',
        'script_path' => '/opt/llm/orchestrator/duo_loop_engine.py',
        'profiles_dir' => '/var/www/html/KUZCHAT-LLM-DUO/storage/orchestrators',
        'output_dir' => '/var/www/html/KUZCHAT-LLM-DUO/storage/runs',
        'launcher_dir' => '/var/www/html/KUZCHAT-LLM-DUO/storage/launcher',
        'pid_file' => '/var/www/html/KUZCHAT-LLM-DUO/storage/launcher/current_run.pid',
        'meta_file' => '/var/www/html/KUZCHAT-LLM-DUO/storage/launcher/current_run.json',
    ],
];
```

`public/index.php`

Single-page HTML shell served on every request. PHP resolves all config values at render time and injects them into `window.KUZCHAT_CONFIG` so the JavaScript frontend can consume them without additional API calls.

HTML shell - loads config, renders all UI panels, injects JS config.

```
<?php

declare(strict_types=1);

$config = require __DIR__ . '/../app/config.php';

date_default_timezone_set($config['timezone']);

$appName = $config['app_name'];
$brandLine = $config['ui']['brand_line'];
$subtitle = $config['ui']['subtitle'];
$pollIntervalMs = (int) $config['ui']['poll_interval_ms'];

$kuzai = $config['nodes']['kuzai'];
$darkai = $config['nodes']['darkai'];

function e(string $value): string
{
    return htmlspecialchars($value, ENT_QUOTES | ENT_SUBSTITUTE, 'UTF-8');
}
?>
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title><?= e($appName) ?></title>
    <link rel="stylesheet" href="/assets/css/style.css">
</head>
<body>
    <div class="app-shell">
        <header class="topbar">
            <div class="topbar__left">
                <div class="logo-box">KUZ</div>
                <div class="title-group">
                    <h1><?= e($appName) ?></h1>
                    <p class="brand-line"><?= e($brandLine) ?></p>
                </div>
            </div>
            <div class="topbar__right">
```

```
                <div class="status-pill" id="global-status-pill">
                    <span class="status-dot"></span>
                    <span id="global-status-text">CHECKING</span>
                </div>
            </div>
        </header>

        <main class="dashboard">
            <section class="hero-card">
                <p class="eyebrow"><?= e($subtitle) ?></p>
                <h2>KUZAI / DARKAI - MONITORING</h2>
                <p class="hero-text">
                    Real-time monitoring of the web server, LLM endpoints, and connectivity
between both nodes.
                    Orchestrator profiles can be edited, saved, and selected before launching
a run.
                </p>
                <div class="hero-stats">
                    <div class="hero-stat">
                        <span class="hero-stat__label">ENGINE</span>
                        <span class="hero-stat__value" id="engine-status">CHECKING</span>
                    </div>
                    <div class="hero-stat">
                        <span class="hero-stat__label">WEB</span>
                        <span class="hero-stat__value" id="web-status">CHECKING</span>
                    </div>
                    <div class="hero-stat">
                        <span class="hero-stat__label">UPDATED</span>
                        <span class="hero-stat__value" id="last-update">--</span>
                    </div>
                </div>
            </section>

            <!-- STATUS SECTION -->
            <section class="section-block">
                <div class="section-header">
                    <p class="section-kicker">STATUS</p>
                    <h3 class="section-title">INFRASTRUCTURE</h3>
                </div>
```

```
                <div class="two-col-grid">
                    <!-- KUZAI NODE CARD -->
                    <article class="node-card node-card--kuzai">
                        <div class="node-card__header">
                            <div>
                                <p class="node-card__role"><?= e($kuzai['role']) ?></p>
                                <h3><?= e($kuzai['label']) ?></h3>
                            </div>
                            <span class="node-badge" id="badge-kuzai">CHECKING</span>
                        </div>
                        <div class="node-metrics">
                            <div class="metric-box">
                                <span class="metric-box__label">HOST</span>
                                <span class="metric-box__value"><?= e($kuzai['host'])
?></span>
                            </div>
                            <div class="metric-box">
                                <span class="metric-box__label">IP</span>
                                <span class="metric-box__value"><?= e($kuzai['ip']) ?></span>
                            </div>
                            <div class="metric-box metric-box--full">
                                <span class="metric-box__label">API BASE</span>
                                <span class="metric-box__value metric-box__value--small"><?=
e($kuzai['api_base']) ?></span>
                            </div>
                            <div class="metric-box metric-box--full">
                                <span class="metric-box__label">MODEL</span>
                                <span class="metric-box__value metric-box__value--small"
id="kuzai-model">--</span>
                            </div>
                            <div class="metric-box">
                                <span class="metric-box__label">HTTP</span>
                                <span class="metric-box__value" id="kuzai-http">--</span>
                            </div>
                            <div class="metric-box">
                                <span class="metric-box__label">LATENCY</span>
                                <span class="metric-box__value" id="kuzai-latency">--</span>
                            </div>
                        </div>
                    </article>
```

```
                    <!-- DARKAI NODE CARD -->
                    <article class="node-card node-card--darkai">
                        <div class="node-card__header">
                            <div>
                                <p class="node-card__role"><?= e($darkai['role']) ?></p>
                                <h3><?= e($darkai['label']) ?></h3>
                            </div>
                            <span class="node-badge" id="badge-darkai">CHECKING</span>
                        </div>
                        <div class="node-metrics">
                            <div class="metric-box">
                                <span class="metric-box__label">HOST</span>
                                <span class="metric-box__value"><?= e($darkai['host'])
?></span>
                            </div>
                            <div class="metric-box">
                                <span class="metric-box__label">IP</span>
                                <span class="metric-box__value"><?= e($darkai['ip']) ?></span>
                            </div>
                            <div class="metric-box metric-box--full">
                                <span class="metric-box__label">API BASE</span>
                                <span class="metric-box__value metric-box__value--small"><?=
e($darkai['api_base']) ?></span>
                            </div>
                            <div class="metric-box metric-box--full">
                                <span class="metric-box__label">MODEL</span>
                                <span class="metric-box__value metric-box__value--small"
id="darkai-model">--</span>
                            </div>
                            <div class="metric-box">
                                <span class="metric-box__label">HTTP</span>
                                <span class="metric-box__value" id="darkai-http">--</span>
                            </div>
                            <div class="metric-box">
                                <span class="metric-box__label">LATENCY</span>
                                <span class="metric-box__value" id="darkai-latency">--</span>
                            </div>
                        </div>
                    </article>
```

```
                </div>
            </section>

            <!-- DISCUSSION / RUN SECTION -->
            <section class="section-block">
                <!-- Orchestrator editor + run control + transcript panels -->
                <article class="panel panel--run-control">
                    <div class="panel__header panel__header--run">
                        <div>
                            <p class="eyebrow">RUN CONTROL</p>
                            <h3>Start a Discussion</h3>
                        </div>
                        <div class="run-header-side">
                            <div class="run-state-pill state-ready" id="run-state-
pill">READY</div>
                            <div class="run-profile-inline">
                                PROFILE --> <span id="run-orchestrator-inline">--</span>
                            </div>
                        </div>
                    </div>

                    <form class="run-form" method="post" action="#" onsubmit="return false;"
id="run-form">
                        <div class="form-row">
                            <label for="opening_prompt">Opening Prompt</label>
                            <textarea id="opening_prompt" name="opening_prompt"
rows="7">Introduce yourself briefly, say your name, and say your purpose in this
project.</textarea>
                        </div>
                        <div class="form-actions form-actions--run">
                            <button type="button" class="btn btn--primary btn--run-main"
id="start-run-btn">START RUN</button>
                            <button type="button" class="btn btn--secondary" id="stop-run-btn"
disabled>STOP</button>
                            <button type="button" class="btn btn--secondary" id="reset-run-
btn">RELOAD</button>
                        </div>
                        <div class="run-status-grid">
                            <div class="metric-box">
                                <span class="metric-box__label">RUN STATE</span>
```

```
                                <span class="metric-box__value" id="run-state">IDLE</span>
                            </div>
                            <div class="metric-box">
                                <span class="metric-box__label">PID</span>
                                <span class="metric-box__value" id="run-pid">--</span>
                            </div>
                            <div class="metric-box">
                                <span class="metric-box__label">ORCHESTRATOR</span>
                                <span class="metric-box__value" id="run-orchestrator">--
</span>
                            </div>
                            <div class="metric-box metric-box--full">
                                <span class="metric-box__label">STARTED AT</span>
                                <span class="metric-box__value metric-box__value--small"
id="run-started">--</span>
                            </div>
                        </div>
                        <p class="panel-note panel-note--run" id="run-note">
                            Ready to launch a Duo run from the selected orchestrator profile.
                        </p>
                    </form>
                </article>

                <!-- LIVE TRANSCRIPT -->
                <article class="panel panel--transcript">
                    <div class="panel__header">
                        <p class="eyebrow">LIVE TRANSCRIPT</p>
                        <h3>Preview</h3>
                    </div>
                    <div class="transcript-list transcript-list--live" id="transcript-live">
                        <div class="message-card message-card--kuzai">
                            <div class="message-card__meta">
                                <span class="message-card__speaker">SYSTEM</span>
                            </div>
                            <div class="message-card__body">
                                No run has been launched yet from the web interface.
                            </div>
                        </div>
                    </div>
                </article>
```

```
            </section>
        </main>
    </div>

    <script>
        window.KUZCHAT_CONFIG = {
            pollIntervalMs: <?= $pollIntervalMs ?>,
            statusUrl: '/api/status.php',
            runStartUrl: '/api/run-start.php',
            runStatusUrl: '/api/run-status.php',
            runStopUrl: '/api/run-stop.php',
            orchestratorListUrl: '/api/orchestrators-list.php',
            orchestratorReadUrl: '/api/orchestrator-read.php',
            orchestratorSaveUrl: '/api/orchestrator-save.php'
        };
    </script>
    <script src="/assets/js/app.js"></script>
</body>
</html>
```

`public/api/status.php`

Called every 5 seconds by the frontend. Reads local system metrics from `/proc` and polls both node APIs via `curl`.
Returns a unified JSON payload covering system health, node reachability, loaded model names, and latency.

Reads system metrics (`/proc/meminfo`, `/proc/uptime`, `/proc/loadavg`) and polls both node APIs via `curl`. Returns a unified JSON payload to the frontend.

```
<?php


declare(strict_types=1);


$config = require __DIR__ . '/../../app/config.php';


date_default_timezone_set($config['timezone']);
header('Content-Type: application/json; charset=UTF-8');


function readTextFileSafe(string $path): ?string
{
```

```
    if (!is_readable($path)) { return null; }
    $content = @file_get_contents($path);
    return $content === false ? null : trim($content);
}

function readHostname(): string
{
    $hostname = readTextFileSafe('/etc/hostname');
    return ($hostname !== null && $hostname !== '') ? $hostname : php_uname('n');
}

function readUptimeSeconds(): ?int
{
    $raw = readTextFileSafe('/proc/uptime');
    if ($raw === null) { return null; }
    $parts = preg_split('/\s+/', $raw);
    return (!isset($parts[0]) || !is_numeric($parts[0])) ? null : (int) floor((float)
$parts[0]);
}

function formatUptime(?int $seconds): string
{
    if ($seconds === null || $seconds < 0) { return 'unknown'; }
    $days = intdiv($seconds, 86400); $seconds %= 86400;
    $hours = intdiv($seconds, 3600); $seconds %= 3600;
    $minutes = intdiv($seconds, 60);
    $parts = [];
    if ($days > 0) { $parts[] = $days . 'd'; }
    if ($hours > 0) { $parts[] = $hours . 'h'; }
    $parts[] = $minutes . 'm';
    return implode(' ', $parts);
}

function readMemoryInfo(): array
{
    $result = ['total_mb' => null, 'available_mb' => null, 'used_mb' => null, 'used_percent'
=> null];
    $raw = readTextFileSafe('/proc/meminfo');
    if ($raw === null) { return $result; }
    $data = [];
```

```
    foreach (explode("\n", $raw) as $line) {
        if (preg_match('/^([A-Za-z_]+):\s+(\d+)\s+kB$/', trim($line), $matches)) {
            $data[$matches[1]] = (int) $matches[2];
        }
    }
    if (!isset($data['MemTotal'], $data['MemAvailable'])) { return $result; }
    $totalKb = $data['MemTotal']; $availableKb = $data['MemAvailable']; $usedKb = $totalKb -
$availableKb;
    $result['total_mb'] = (int) round($totalKb / 1024);
    $result['available_mb'] = (int) round($availableKb / 1024);
    $result['used_mb'] = (int) round($usedKb / 1024);
    if ($totalKb > 0) { $result['used_percent'] = round(($usedKb / $totalKb) * 100, 1); }
    return $result;
}

function readLoadAverage(): ?string
{
    $raw = readTextFileSafe('/proc/loadavg');
    if ($raw === null) { return null; }
    $parts = preg_split('/\s+/', $raw);
    return (count($parts) < 3) ? null : $parts[0] . ' / ' . $parts[1] . ' / ' . $parts[2];
}

function httpJson(string $url, int $timeoutSeconds = 4): array
{
    $ch = curl_init($url);
    curl_setopt_array($ch, [
        CURLOPT_RETURNTRANSFER => true, CURLOPT_FOLLOWLOCATION => false,
        CURLOPT_CONNECTTIMEOUT => $timeoutSeconds, CURLOPT_TIMEOUT => $timeoutSeconds,
        CURLOPT_HTTPHEADER => ['Accept: application/json'],
    ]);
    $start = microtime(true);
    $body = curl_exec($ch);
    $latencyMs = (int) round((microtime(true) - $start) * 1000);
    $errno = curl_errno($ch);
    $error = $errno !== 0 ? curl_error($ch) : null;
    $status = (int) curl_getinfo($ch, CURLINFO_RESPONSE_CODE);
    curl_close($ch);
    if ($errno !== 0) {
        return ['ok' => false, 'http_code' => $status, 'latency_ms' => $latencyMs, 'error' =>
```

```
$error, 'data' => null];
    }
    if ($body === false || $body === '') {
        return ['ok' => false, 'http_code' => $status, 'latency_ms' => $latencyMs, 'error' =>
'Empty response body', 'data' => null];
    }
    $decoded = json_decode($body, true);
    if (!is_array($decoded)) {
        return ['ok' => false, 'http_code' => $status, 'latency_ms' => $latencyMs, 'error' =>
'Invalid JSON response', 'data' => null];
    }
    return ['ok' => $status >= 200 && $status < 300, 'http_code' => $status, 'latency_ms' =>
$latencyMs, 'error' => null, 'data' => $decoded];
}

function extractModelName(?array $payload): ?string
{
    if ($payload === null) { return null; }
    if (isset($payload['data'][0]['id']) && is_string($payload['data'][0]['id'])) { return
$payload['data'][0]['id']; }
    if (isset($payload['models'][0]['name']) && is_string($payload['models'][0]['name'])) {
return $payload['models'][0]['name']; }
    return null;
}

function tcpReachable(string $host, int $port, float $timeoutSeconds = 2.0): array
{
    $errno = 0; $errstr = '';
    $start = microtime(true);
    $fp = @fsockopen($host, $port, $errno, $errstr, $timeoutSeconds);
    $latencyMs = (int) round((microtime(true) - $start) * 1000);
    if (is_resource($fp)) { fclose($fp); return ['ok' => true, 'latency_ms' => $latencyMs,
'error' => null]; }
    return ['ok' => false, 'latency_ms' => $latencyMs, 'error' => trim($errstr) !== '' ?
$errstr : 'Connection failed'];
}

$kuzaiModels = httpJson($config['nodes']['kuzai']['models_endpoint']);
$darkaiModels = httpJson($config['nodes']['darkai']['models_endpoint']);
$darkaiTcp = tcpReachable($config['nodes']['darkai']['ip'], 8080);
```

```
$memory = readMemoryInfo();
$uptimeSeconds = readUptimeSeconds();

$response = [
    'app' => [
        'name' => $config['app_name'],
        'timestamp' => date('Y-m-d H:i:s'),
        'php_version' => PHP_VERSION,
        'apache_hint' => $_SERVER['SERVER_SOFTWARE'] ?? 'unknown',
    ],
    'system' => [
        'hostname' => readHostname(),
        'server_ip' => $_SERVER['SERVER_ADDR'] ?? $config['nodes']['kuzai']['ip'],
        'load_average' => readLoadAverage(),
        'uptime' => formatUptime($uptimeSeconds),
        'memory' => $memory,
    ],
    'nodes' => [
        'kuzai' => [
            'label' => $config['nodes']['kuzai']['label'],
            'host' => $config['nodes']['kuzai']['host'],
            'ip' => $config['nodes']['kuzai']['ip'],
            'reachable' => $kuzaiModels['ok'],
            'http_code' => $kuzaiModels['http_code'],
            'latency_ms' => $kuzaiModels['latency_ms'],
            'model_name' => extractModelName($kuzaiModels['data']),
            'error' => $kuzaiModels['error'],
        ],
        'darkai' => [
            'label' => $config['nodes']['darkai']['label'],
            'host' => $config['nodes']['darkai']['host'],
            'ip' => $config['nodes']['darkai']['ip'],
            'reachable' => $darkaiModels['ok'],
            'http_code' => $darkaiModels['http_code'],
            'latency_ms' => $darkaiModels['latency_ms'],
            'model_name' => extractModelName($darkaiModels['data']),
            'error' => $darkaiModels['error'],
            'tcp_8080' => $darkaiTcp,
        ],
    ],
```

```
    'checks' => [
        'web_php' => true,
        'kuzai_models_api' => $kuzaiModels['ok'],
        'darkai_models_api' => $darkaiModels['ok'],
        'fhc2_to_fhc_tcp_8080' => $darkaiTcp['ok'],
    ],
];

http_response_code(200);
echo json_encode($response, JSON_PRETTY_PRINT | JSON_UNESCAPED_SLASHES |
JSON_UNESCAPED_UNICODE);
```

`public/api/run-start.php`

Receives the opening prompt and selected orchestrator profile slug. Validates all paths, checks that no run is already active via `/proc/<PID>`, then launches the Python orchestrator as a background process with `nohup`. Returns the PID and start time.

Validates the incoming JSON POST, checks that the Python binary and orchestrator profile exist, ensures no run is already active (via `/proc/<PID>`), launches the process with `nohup`, writes PID and metadata files.

```
<?php

declare(strict_types=1);

$config = require __DIR__ . '/../../app/config.php';

date_default_timezone_set($config['timezone']);
header('Content-Type: application/json; charset=UTF-8');

if ($_SERVER['REQUEST_METHOD'] !== 'POST') {
    http_response_code(405);
    echo json_encode(['ok' => false, 'error' => 'Method not allowed']);
    exit;
}

function sanitizeSlug(string $value): string
{
    $value = strtolower(trim($value));
    $value = preg_replace('/[^a-z0-9._-]+/', '-', $value) ?? '';
```

```
    return trim($value, '-._');
}

$rawInput = file_get_contents('php://input');
$data = json_decode($rawInput ?: '', true);

if (!is_array($data)) {
    http_response_code(400);
    echo json_encode(['ok' => false, 'error' => 'Invalid JSON body']);
    exit;
}

$openingPrompt = trim((string) ($data['opening_prompt'] ?? ''));
$orchestratorName = sanitizeSlug((string) ($data['orchestrator_name'] ?? ''));

if ($openingPrompt === '') {
    http_response_code(422);
    echo json_encode(['ok' => false, 'error' => 'Opening prompt is required']);
    exit;
}

if ($orchestratorName === '') {
    http_response_code(422);
    echo json_encode(['ok' => false, 'error' => 'Orchestrator name is required']);
    exit;
}

$pythonBin   = $config['orchestrator']['python_bin'];
$scriptPath  = $config['orchestrator']['script_path'];
$outputDir   = $config['orchestrator']['output_dir'];
$launcherDir = $config['orchestrator']['launcher_dir'];
$pidFile     = $config['orchestrator']['pid_file'];
$metaFile    = $config['orchestrator']['meta_file'];
$profileFile = $config['orchestrator']['profiles_dir'] . '/' . $orchestratorName . '.json';

foreach ([$launcherDir, $outputDir] as $dir) {
    if (!is_dir($dir) && !@mkdir($dir, 0775, true)) {
        http_response_code(500);
        echo json_encode(['ok' => false, 'error' => 'Unable to create directory: ' . $dir]);
        exit;
```

```
    }
}

if (!is_executable($pythonBin)) {
    http_response_code(500);
    echo json_encode(['ok' => false, 'error' => 'Python binary not executable: ' .
$pythonBin]);
    exit;
}

if (!is_file($scriptPath)) {
    http_response_code(500);
    echo json_encode(['ok' => false, 'error' => 'Engine not found: ' . $scriptPath]);
    exit;
}

if (!is_file($profileFile)) {
    http_response_code(404);
    echo json_encode(['ok' => false, 'error' => 'Orchestrator profile not found']);
    exit;
}

// Block concurrent runs
if (is_file($pidFile)) {
    $rawPid = trim((string) @file_get_contents($pidFile));
    if ($rawPid !== '' && ctype_digit($rawPid) && is_dir('/proc/' . $rawPid)) {
        http_response_code(409);
        echo json_encode(['ok' => false, 'error' => 'A run is already in progress', 'pid' =>
(int) $rawPid]);
        exit;
    }
}

$timestamp   = date('Ymd-His');
$stdoutFile  = $launcherDir . '/run-' . $timestamp . '.stdout.log';
$stderrFile  = $launcherDir . '/run-' . $timestamp . '.stderr.log';

$command = implode(' ', [
    'nohup',
    escapeshellarg($pythonBin),
```

```
    escapeshellarg($scriptPath),
    '--profile-file', escapeshellarg($profileFile),
    '--opening-prompt', escapeshellarg($openingPrompt),
    '--output-dir', escapeshellarg($outputDir),
    '>', escapeshellarg($stdoutFile),
    '2>', escapeshellarg($stderrFile),
    '& echo $!',
]);

$output = []; $returnCode = 0;
exec($command, $output, $returnCode);

$pid = null;
if (isset($output[0])) {
    $pidRaw = trim((string) $output[0]);
    if ($pidRaw !== '' && ctype_digit($pidRaw)) { $pid = (int) $pidRaw; }
}

if ($returnCode !== 0 || $pid === null || $pid < 1) {
    http_response_code(500);
    echo json_encode(['ok' => false, 'error' => 'Unable to start process', 'return_code' =>
$returnCode]);
    exit;
}

@file_put_contents($pidFile, (string) $pid);

$meta = [
    'pid' => $pid, 'status' => 'running', 'started_at' => date('Y-m-d H:i:s'),
    'opening_prompt' => $openingPrompt, 'orchestrator_name' => $orchestratorName,
    'profile_file' => $profileFile, 'stdout_file' => $stdoutFile, 'stderr_file' =>
$stderrFile,
    'output_dir' => $outputDir,
];
@file_put_contents($metaFile, json_encode($meta, JSON_PRETTY_PRINT | JSON_UNESCAPED_UNICODE));

http_response_code(200);
echo json_encode(['ok' => true, 'message' => 'Run started', 'pid' => $pid,
    'started_at' => $meta['started_at'], 'orchestrator_name' => $orchestratorName]);
```

`public/api/run-status.php`

Polled by the frontend every 5 seconds. Checks process liveness in `/proc`, reads the latest transcript from the most recent `run-ab-*` directory, and transitions the status machine (`running` → `completed` / `failed` / `stopped`). Returns the full transcript entries and log tails.

Checks `/proc/<PID>` for liveness. Reads `transcript.json` from the latest `run-ab-*` directory. Updates the meta file with the current state (`running`, `completed`, `stopped`, `failed`). Returns status, transcript entries, and stdout/stderr tails.

```
<?php

declare(strict_types=1);

$config = require __DIR__ . '/../../app/config.php';

date_default_timezone_set($config['timezone']);
header('Content-Type: application/json; charset=UTF-8');

function readJsonFileSafe(string $path): ?array
{
    if (!is_file($path) || !is_readable($path)) { return null; }
    $raw = @file_get_contents($path);
    if ($raw === false || trim($raw) === '') { return null; }
    $decoded = json_decode($raw, true);
    return is_array($decoded) ? $decoded : null;
}

function writeJsonFileSafe(string $path, array $data): void
{
    @file_put_contents($path, json_encode($data, JSON_PRETTY_PRINT | JSON_UNESCAPED_UNICODE));
}

function tailFileSafe(string $path, int $maxLines = 20): array
{
    if (!is_file($path) || !is_readable($path)) { return []; }
    $lines = @file($path, FILE_IGNORE_NEW_LINES);
    return !is_array($lines) ? [] : array_slice($lines, -$maxLines);
}

function findLatestRun(string $runsDir): ?array
```

```
{
    if (!is_dir($runsDir)) { return null; }
    $entries = scandir($runsDir);
    if (!is_array($entries)) { return null; }
    $dirs = [];
    foreach ($entries as $entry) {
        if ($entry === '.' || $entry === '..') { continue; }
        $fullPath = $runsDir . '/' . $entry;
        if (is_dir($fullPath) && str_starts_with($entry, 'run-ab-')) {
            $dirs[] = ['name' => $entry, 'path' => $fullPath, 'mtime' => @filemtime($fullPath)
?: 0];
        }
    }
    if ($dirs === []) { return null; }
    usort($dirs, static fn(array $a, array $b): int => $b['mtime'] <=> $a['mtime']);
    return $dirs[0];
}

function readTranscriptEntries(string $runDir): array
{
    $decoded = readJsonFileSafe($runDir . '/transcript.json');
    if (!is_array($decoded)) { return []; }
    $entries = [];
    foreach ($decoded as $item) {
        if (!is_array($item)) { continue; }
        $entries[] = [
            'turn' => isset($item['turn']) ? (int) $item['turn'] : 0,
            'speaker' => (string) ($item['speaker'] ?? ''),
            'content' => (string) ($item['content'] ?? ''),
            'timestamp' => (string) ($item['timestamp'] ?? ''),
        ];
    }
    return $entries;
}

$pidFile  = $config['orchestrator']['pid_file'];
$metaFile = $config['orchestrator']['meta_file'];
$runsDir  = $config['orchestrator']['output_dir'];

$meta = readJsonFileSafe($metaFile);
```

```
$latestRun = findLatestRun($runsDir);

$pid = null;
if (is_file($pidFile)) {
    $rawPid = trim((string) @file_get_contents($pidFile));
    if ($rawPid !== '' && ctype_digit($rawPid)) { $pid = (int) $rawPid; }
}

$isRunning = $pid !== null && $pid > 0 && is_dir('/proc/' . $pid);
$transcript = [];
$latestRunInfo = null;

if ($latestRun !== null) {
    $transcript = readTranscriptEntries($latestRun['path']);
    $latestRunInfo = [
        'name' => $latestRun['name'], 'path' => $latestRun['path'],
        'mtime' => date('Y-m-d H:i:s', (int) $latestRun['mtime']),
        'transcript_json' => is_file($latestRun['path'] . '/transcript.json') ?
$latestRun['path'] . '/transcript.json' : null,
        'transcript_md'   => is_file($latestRun['path'] . '/transcript.md')   ?
$latestRun['path'] . '/transcript.md'   : null,
    ];
}

$stdoutTail = []; $stderrTail = [];
if ($meta !== null) {
    $stdoutTail = tailFileSafe((string) ($meta['stdout_file'] ?? ''), 30);
    $stderrTail = tailFileSafe((string) ($meta['stderr_file'] ?? ''), 30);
}

// Determine state
if ($isRunning) {
    $status = 'running'; $note = 'A Duo run is currently in progress.';
    if ($meta !== null && ($meta['status'] ?? '') !== 'running') {
        $meta['status'] = 'running'; writeJsonFileSafe($metaFile, $meta);
    }
} elseif ($meta !== null && $latestRunInfo !== null && count($transcript) > 0) {
    $status = 'completed'; $note = 'The last Duo run completed successfully.';
    @unlink($pidFile);
    $meta['status'] = 'completed';
```

```
    if (!isset($meta['completed_at'])) { $meta['completed_at'] = date('Y-m-d H:i:s'); }
    $meta['latest_run'] = $latestRunInfo['name']; writeJsonFileSafe($metaFile, $meta);
} elseif ($meta !== null && (($meta['status'] ?? '') === 'stopped')) {
    $status = 'stopped'; $note = 'The last Duo run was stopped manually.';
    @unlink($pidFile); writeJsonFileSafe($metaFile, $meta);
} elseif ($meta !== null && $stderrTail !== []) {
    $status = 'failed'; $note = 'The last run ended with an error.';
    @unlink($pidFile);
    $meta['status'] = 'failed';
    if (!isset($meta['completed_at'])) { $meta['completed_at'] = date('Y-m-d H:i:s'); }
    writeJsonFileSafe($metaFile, $meta);
} else {
    $status = 'idle'; $note = 'No run launched yet.';
}

http_response_code(200);
echo json_encode([
    'ok' => true, 'status' => $status, 'note' => $note, 'active' => $meta,
    'is_running' => $isRunning, 'pid' => $isRunning ? $pid : null,
    'latest_run' => $latestRunInfo, 'transcript' => $transcript,
    'stdout_tail' => $stdoutTail, 'stderr_tail' => $stderrTail,
    'timestamp' => date('Y-m-d H:i:s'),
], JSON_PRETTY_PRINT | JSON_UNESCAPED_UNICODE);
```

`public/api/run-stop.php`

Sends `SIGTERM` to the active PID and waits 300 ms. If the process is still alive, sends `SIGKILL`. Cleans up the PID file and updates the meta file with a `stopped` status and stop timestamp.

Sends `SIGTERM` to the PID. Waits 300 ms. If still alive, sends `SIGKILL`. Removes PID file and writes `stopped` status to meta file.

```
<?php

declare(strict_types=1);

$config = require __DIR__ . '/../../app/config.php';

date_default_timezone_set($config['timezone']);
header('Content-Type: application/json; charset=UTF-8');
```

```
if ($_SERVER['REQUEST_METHOD'] !== 'POST') {
    http_response_code(405);
    echo json_encode(['ok' => false, 'error' => 'Method not allowed']);
    exit;
}

$pidFile  = $config['orchestrator']['pid_file'];
$metaFile = $config['orchestrator']['meta_file'];

$pid = null;
if (is_file($pidFile)) {
    $rawPid = trim((string) @file_get_contents($pidFile));
    if ($rawPid !== '' && ctype_digit($rawPid)) { $pid = (int) $rawPid; }
}

if ($pid === null || $pid < 1) {
    http_response_code(200);
    echo json_encode(['ok' => true, 'message' => 'No active run']);
    exit;
}

if (!is_dir('/proc/' . $pid)) {
    @unlink($pidFile);
    http_response_code(200);
    echo json_encode(['ok' => true, 'message' => 'Run already finished', 'pid' => $pid]);
    exit;
}

exec('kill ' . escapeshellarg((string) $pid), $output, $returnCode);
usleep(300000);
$stillRunning = is_dir('/proc/' . $pid);

if ($stillRunning) {
    exec('kill -9 ' . escapeshellarg((string) $pid));
    usleep(200000);
    $stillRunning = is_dir('/proc/' . $pid);
}

if ($stillRunning) {
    http_response_code(500);
```

```
    echo json_encode(['ok' => false, 'error' => 'Unable to stop process', 'pid' => $pid]);
    exit;
}

@unlink($pidFile);

$meta = [];
if (is_file($metaFile) && is_readable($metaFile)) {
    $decoded = json_decode((string) @file_get_contents($metaFile), true);
    if (is_array($decoded)) { $meta = $decoded; }
}

$meta['status'] = 'stopped';
$meta['stopped_at'] = date('Y-m-d H:i:s');
$meta['stopped_pid'] = $pid;
@file_put_contents($metaFile, json_encode($meta, JSON_PRETTY_PRINT | JSON_UNESCAPED_UNICODE));

http_response_code(200);
echo json_encode(['ok' => true, 'message' => 'Run stopped', 'pid' => $pid]);
```

`public/api/orchestrators-list.php`

Scans `storage/orchestrators/` for `.json` files and returns a sorted list of profile names, slugs, descriptions, and last-modified dates. Used by the frontend to populate the profile selector.

Scans the `storage/orchestrators/` directory for `.json` files. Returns name, slug, description, and last modified date for each profile, sorted alphabetically.

```
<?php

declare(strict_types=1);

$config = require __DIR__ . '/../../app/config.php';

date_default_timezone_set($config['timezone']);
header('Content-Type: application/json; charset=UTF-8');

$profilesDir = $config['orchestrator']['profiles_dir'];

if (!is_dir($profilesDir)) { @mkdir($profilesDir, 0775, true); }
```

```
$items = [];
$files = glob($profilesDir . '/*.json') ?: [];

foreach ($files as $file) {
    $slug = basename($file, '.json');
    $raw = @file_get_contents($file);
    $decoded = is_string($raw) ? json_decode($raw, true) : null;
    $items[] = [
        'slug' => $slug,
        'name' => is_array($decoded) && isset($decoded['name']) ? (string) $decoded['name'] :
$slug,
        'description' => is_array($decoded) && isset($decoded['description']) ? (string)
$decoded['description'] : '',
        'updated_at' => date('Y-m-d H:i:s', (int) (@filemtime($file) ?: time())),
    ];
}

usort($items, static fn(array $a, array $b): int => strcasecmp($a['name'], $b['name']));

echo json_encode(['ok' => true, 'items' => $items, 'count' => count($items)],
    JSON_PRETTY_PRINT | JSON_UNESCAPED_UNICODE);
```

`public/api/orchestrator-read.php`

Loads a single orchestrator profile by slug from `storage/orchestrators/`. All fields are normalized and clamped to valid ranges before being returned. Returns 404 if the slug does not match any file.

Reads a profile JSON file by slug from `storage/orchestrators/`. Normalizes and validates all fields before returning. Returns 404 if not found.

```
<?php

declare(strict_types=1);

$config = require __DIR__ . '/../../app/config.php';

date_default_timezone_set($config['timezone']);
header('Content-Type: application/json; charset=UTF-8');

function sanitizeSlug(string $value): string
```

```
{
    $value = strtolower(trim($value));
    $value = preg_replace('/[^a-z0-9._-]+/', '-', $value) ?? '';
    return trim($value, '-._');
}

function normalizeProfile(array $profile, string $slug): array
{
    return [
        'slug' => $slug,
        'name' => trim((string) ($profile['name'] ?? $slug)) !== '' ? trim((string)
($profile['name'] ?? $slug)) : $slug,
        'description' => trim((string) ($profile['description'] ?? '')),
        'run' => [
            'turns'         => max(1,  min(40,   (int)   ($profile['run']['turns']         ??
6))),
            'max_lines'     => max(1,  min(20,   (int)   ($profile['run']['max_lines']     ??
5))),
            'max_chars'     => max(50, min(4000, (int)   ($profile['run']['max_chars']     ??
500))),
            'history_depth' => max(0,  min(20,   (int)   ($profile['run']['history_depth'] ??
3))),
            'max_sentences' => max(1,  min(12,   (int)   ($profile['run']['max_sentences'] ??
4))),
        ],
        'kuzai' => [
            'label'           => 'KUZAI',
            'system_prompt'   => (string) ($profile['kuzai']['system_prompt']   ?? 'You are
KUZAI. Reply clearly and stay concise.'),
            'temperature'     => (float)  ($profile['kuzai']['temperature']     ?? 0.35),
            'top_p'           => (float)  ($profile['kuzai']['top_p']           ?? 0.95),
            'top_k'           => (int)    ($profile['kuzai']['top_k']           ?? 40),
            'max_tokens'      => (int)    ($profile['kuzai']['max_tokens']      ?? 300),
            'repeat_penalty'  => (float)  ($profile['kuzai']['repeat_penalty']  ?? 1.05),
        ],
        'darkai' => [
            'label'           => 'DARKAI',
            'system_prompt'   => (string) ($profile['darkai']['system_prompt']   ?? 'You are
DARKAI. Reply clearly and stay concise.'),
            'temperature'     => (float)  ($profile['darkai']['temperature']     ?? 0.35),
```

```
            'top_p'           => (float)  ($profile['darkai']['top_p']           ?? 0.95),
            'top_k'           => (int)    ($profile['darkai']['top_k']           ?? 40),
            'max_tokens'      => (int)    ($profile['darkai']['max_tokens']      ?? 300),
            'repeat_penalty'  => (float)  ($profile['darkai']['repeat_penalty']  ?? 1.05),
        ],
    ];
}

$slug = sanitizeSlug((string) ($_GET['name'] ?? ''));

if ($slug === '') {
    http_response_code(422);
    echo json_encode(['ok' => false, 'error' => 'Missing orchestrator name']);
    exit;
}

$file = $config['orchestrator']['profiles_dir'] . '/' . $slug . '.json';

if (!is_file($file) || !is_readable($file)) {
    http_response_code(404);
    echo json_encode(['ok' => false, 'error' => 'Orchestrator not found']);
    exit;
}

$raw = @file_get_contents($file);
$decoded = is_string($raw) ? json_decode($raw, true) : null;

if (!is_array($decoded)) {
    http_response_code(500);
    echo json_encode(['ok' => false, 'error' => 'Invalid orchestrator file']);
    exit;
}

echo json_encode(['ok' => true, 'profile' => normalizeProfile($decoded, $slug)],
    JSON_PRETTY_PRINT | JSON_UNESCAPED_UNICODE);
```

`public/api/orchestrator-save.php`

Writes or overwrites an orchestrator profile JSON file in `storage/orchestrators/`. The slug is derived from the profile name by sanitization. All numeric fields are clamped to defined min/max ranges on

write.

Accepts a POST with a `profile` JSON object. Sanitizes the slug, normalizes all fields (clamped ranges), writes the profile to `storage/orchestrators/<slug>.json`.

```
<?php

declare(strict_types=1);

$config = require __DIR__ . '/../../app/config.php';

date_default_timezone_set($config['timezone']);
header('Content-Type: application/json; charset=UTF-8');

if ($_SERVER['REQUEST_METHOD'] !== 'POST') {
    http_response_code(405);
    echo json_encode(['ok' => false, 'error' => 'Method not allowed']);
    exit;
}

function sanitizeSlug(string $value): string
{
    $value = strtolower(trim($value));
    $value = preg_replace('/[^a-z0-9._-]+/', '-', $value) ?? '';
    return trim($value, '-._');
}

function normalizeProfile(array $profile, string $slug): array
{
    return [
        'slug' => $slug,
        'name' => trim((string) ($profile['name'] ?? $slug)) !== '' ? trim((string)
($profile['name'] ?? $slug)) : $slug,
        'description' => trim((string) ($profile['description'] ?? '')),
        'run' => [
            'turns'         => max(1,  min(40,   (int)   ($profile['run']['turns']         ??
6))),
            'max_lines'     => max(1,  min(20,   (int)   ($profile['run']['max_lines']     ??
5))),
            'max_chars'     => max(50, min(4000, (int)   ($profile['run']['max_chars']     ??
500))),
```

```
            'history_depth' => max(0,  min(20,   (int)   ($profile['run']['history_depth'] ??
3))),
            'max_sentences' => max(1,  min(12,   (int)   ($profile['run']['max_sentences'] ??
4))),
        ],
        'kuzai' => [
            'label'          => 'KUZAI',
            'system_prompt'  => (string) ($profile['kuzai']['system_prompt']  ?? 'You are
KUZAI. Reply clearly and stay concise.'),
            'temperature'    => (float)  ($profile['kuzai']['temperature']    ?? 0.35),
            'top_p'          => (float)  ($profile['kuzai']['top_p']          ?? 0.95),
            'top_k'          => (int)    ($profile['kuzai']['top_k']          ?? 40),
            'max_tokens'     => (int)    ($profile['kuzai']['max_tokens']     ?? 300),
            'repeat_penalty' => (float)  ($profile['kuzai']['repeat_penalty'] ?? 1.05),
        ],
        'darkai' => [
            'label'          => 'DARKAI',
            'system_prompt'  => (string) ($profile['darkai']['system_prompt']  ?? 'You are
DARKAI. Reply clearly and stay concise.'),
            'temperature'    => (float)  ($profile['darkai']['temperature']    ?? 0.35),
            'top_p'          => (float)  ($profile['darkai']['top_p']          ?? 0.95),
            'top_k'          => (int)    ($profile['darkai']['top_k']          ?? 40),
            'max_tokens'     => (int)    ($profile['darkai']['max_tokens']     ?? 300),
            'repeat_penalty' => (float)  ($profile['darkai']['repeat_penalty'] ?? 1.05),
        ],
    ];
}

$raw = file_get_contents('php://input');
$data = json_decode($raw ?: '', true);

if (!is_array($data) || !isset($data['profile']) || !is_array($data['profile'])) {
    http_response_code(400);
    echo json_encode(['ok' => false, 'error' => 'Invalid JSON body']);
    exit;
}

$requestedName = (string) ($data['save_as_name'] ?? $data['profile']['slug'] ??
$data['profile']['name'] ?? '');
$slug = sanitizeSlug($requestedName);
```

```
if ($slug === '') {
    http_response_code(422);
    echo json_encode(['ok' => false, 'error' => 'Invalid orchestrator name']);
    exit;
}

$profilesDir = $config['orchestrator']['profiles_dir'];
if (!is_dir($profilesDir) && !@mkdir($profilesDir, 0775, true)) {
    http_response_code(500);
    echo json_encode(['ok' => false, 'error' => 'Unable to create orchestrator directory']);
    exit;
}

$profile = normalizeProfile($data['profile'], $slug);
$file = $profilesDir . '/' . $slug . '.json';
$written = @file_put_contents($file, json_encode($profile, JSON_PRETTY_PRINT |
JSON_UNESCAPED_UNICODE) . "\n");

if ($written === false) {
    http_response_code(500);
    echo json_encode(['ok' => false, 'error' => 'Unable to write orchestrator file']);
    exit;
}

echo json_encode(['ok' => true, 'message' => 'Orchestrator saved', 'slug' => $slug, 'profile'
=> $profile],
    JSON_PRETTY_PRINT | JSON_UNESCAPED_UNICODE);
```

`public/assets/js/app.js`

Self-contained IIFE, no framework. Initializes on page load, binds all button events, and starts a polling loop calling `status.php` and `run-status.php` every 5 seconds. Handles the full profile load/save/save-as cycle and renders transcript entries into the live panel.

Frontend controller — IIFE, no framework. Polls `status.php` and `run-status.php` every 5 seconds. Handles load/save/save-as for orchestrator profiles. Manages start/stop/reset run buttons. Renders transcript entries and system messages into `#transcript-live`.

```
(function () {
    "use strict";
```

```
const cfg = window.KUZCHAT_CONFIG || {
    pollIntervalMs: 5000,
    statusUrl: "/api/status.php",
    runStartUrl: "/api/run-start.php",
    runStatusUrl: "/api/run-status.php",
    runStopUrl: "/api/run-stop.php",
    orchestratorListUrl: "/api/orchestrators-list.php",
    orchestratorReadUrl: "/api/orchestrator-read.php",
    orchestratorSaveUrl: "/api/orchestrator-save.php"
};

let currentOrchestratorSlug = null;
let runInterfaceReset = false;

function byId(id) { return document.getElementById(id); }
function setText(id, value) { const el = byId(id); if (el) el.textContent = value; }

function escapeHtml(value) {
    return String(value)
        .replaceAll("&", "&amp;").replaceAll("<", "&lt;").replaceAll(">", "&gt;")
        .replaceAll('"', "&quot;").replaceAll("'", "&#039;");
}

function setBadge(id, ok) {
    const el = byId(id);
    if (!el) return;
    el.textContent = ok ? "ONLINE" : "OFFLINE";
    el.classList.remove("node-badge--ok", "node-badge--down");
    el.classList.add(ok ? "node-badge--ok" : "node-badge--down");
}

function setGlobalStatus(allOk) {
    const text = byId("global-status-text");
    const pill = byId("global-status-pill");
    if (!text || !pill) return;
    text.textContent = allOk ? "READY" : "DEGRADED";
    pill.classList.remove("status-pill--ok", "status-pill--down");
    pill.classList.add(allOk ? "status-pill--ok" : "status-pill--down");
}
```

```
    function applyRunState(state, labelOverride = null) {
        const stateText = byId("run-state");
        const statePill = byId("run-state-pill");
        if (!stateText || !statePill) return;
        const normalized = String(state || "idle").toLowerCase();
        const labels = {
            ready: "READY", idle: "IDLE", running: "RUNNING", completed: "COMPLETED",
            stopped: "STOPPED", error: "ERROR", starting: "STARTING", stopping: "STOPPING",
            reset: "READY FOR NEW RUN", finished: "FINISHED", failed: "FAILED"
        };
        const cssClass = {
            ready: "state-ready", idle: "state-idle", running: "state-running",
            completed: "state-completed", stopped: "state-stopped", error: "state-error",
            starting: "state-running", stopping: "state-stopped", reset: "state-ready",
            finished: "state-completed", failed: "state-error"
        };
        const finalLabel = labelOverride || labels[normalized] || normalized.toUpperCase();
        stateText.textContent = finalLabel;
        statePill.textContent = finalLabel;
        statePill.className = "run-state-pill";
        statePill.classList.add(cssClass[normalized] || "state-idle");
    }

    function renderSystemTranscript(message) {
        const container = byId("transcript-live");
        if (!container) return;
        container.innerHTML = `
            <div class="message-card message-card--kuzai">
                <div class="message-card__meta"><span class="message-
card__speaker">SYSTEM</span></div>
                <div class="message-card__body">${escapeHtml(message)}</div>
            </div>`;
    }

    function renderTranscript(entries, stdoutTail, status, note) {
        const container = byId("transcript-live");
        if (!container) return;
        if (entries.length === 0 && stdoutTail.length === 0) {
            const msg = (status === 'running')
```

```
                ? 'Run in progress. Waiting for first response...'
                : (note || 'No transcript available yet.');
            renderSystemTranscript(msg);
            return;
        }
        const html = entries.map(entry => {
            const isKuzai = String(entry.speaker).toUpperCase().includes('KUZAI');
            const cssClass = isKuzai ? 'message-card--kuzai' : 'message-card--darkai';
            return `
                <div class="message-card ${cssClass}">
                    <div class="message-card__meta">
                        <span class="message-
card__speaker">${escapeHtml(entry.speaker)}</span>
                        <span class="message-card__time">${escapeHtml(entry.timestamp)}</span>
                    </div>
                    <div class="message-card__body">${escapeHtml(entry.content)}</div>
                </div>`;
        }).join('');
        container.innerHTML = html || '';
    }

    function updateRunButtons(isRunning, isStarting, isStopping) {
        const startBtn = byId("start-run-btn");
        const stopBtn  = byId("stop-run-btn");
        const resetBtn = byId("reset-run-btn");
        if (startBtn) {
            startBtn.disabled = isRunning || isStarting || isStopping;
            startBtn.textContent = isStarting ? "STARTING..." : "START RUN";
        }
        if (stopBtn) {
            stopBtn.disabled = !isRunning || isStarting || isStopping;
            stopBtn.textContent = isStopping ? "STOPPING..." : "STOP";
        }
        if (resetBtn) {
            resetBtn.disabled = isRunning || isStarting || isStopping;
            resetBtn.textContent = "RELOAD";
        }
    }

    function syncSelectedOrchestratorDisplay(value) {
```

```
        const finalValue = value || currentOrchestratorSlug || "--";
        setText("run-orchestrator", finalValue);
        setText("run-orchestrator-inline", finalValue);
    }

    function resetRunInterface() {
        runInterfaceReset = true;
        updateRunButtons(false, false, false);
        applyRunState("reset");
        setText("run-pid", "--");
        setText("run-started", "--");
        syncSelectedOrchestratorDisplay(byId("orchestrator_select")?.value ||
currentOrchestratorSlug || "--");
        setText("run-note", "Interface reset. Ready for a new run.");
        renderSystemTranscript("Interface reset. Ready for a new run.");
    }

    function fillProfileForm(profile) {
        currentOrchestratorSlug = profile.slug || null;
        const select = byId("orchestrator_select");
        if (select && profile.slug) select.value = profile.slug;
        byId("profile_name").value        = profile.name        || profile.slug || "";
        byId("profile_slug").value        = profile.slug        || "";
        byId("profile_description").value = profile.description || "";
        byId("profile_turns").value        = profile.run?.turns         ?? 6;
        byId("profile_max_lines").value    = profile.run?.max_lines     ?? 5;
        byId("profile_max_chars").value    = profile.run?.max_chars     ?? 500;
        byId("profile_history_depth").value= profile.run?.history_depth ?? 3;
        byId("kuzai_system_prompt").value  = profile.kuzai?.system_prompt  ?? "";
        byId("kuzai_temperature").value    = profile.kuzai?.temperature    ?? 0.35;
        byId("kuzai_top_p").value          = profile.kuzai?.top_p          ?? 0.95;
        byId("kuzai_top_k").value          = profile.kuzai?.top_k          ?? 40;
        byId("kuzai_max_tokens").value     = profile.kuzai?.max_tokens     ?? 300;
        byId("kuzai_repeat_penalty").value = profile.kuzai?.repeat_penalty ?? 1.05;
        byId("darkai_system_prompt").value  = profile.darkai?.system_prompt  ?? "";
        byId("darkai_temperature").value    = profile.darkai?.temperature    ?? 0.35;
        byId("darkai_top_p").value          = profile.darkai?.top_p          ?? 0.95;
        byId("darkai_top_k").value          = profile.darkai?.top_k          ?? 40;
        byId("darkai_max_tokens").value     = profile.darkai?.max_tokens     ?? 300;
        byId("darkai_repeat_penalty").value = profile.darkai?.repeat_penalty ?? 1.05;
```

```
        if (runInterfaceReset) syncSelectedOrchestratorDisplay(profile.slug || "--");
    }

    async function loadStatus() {
        try {
            const response = await fetch(cfg.statusUrl, { cache: "no-store" });
            const data = await response.json();
            const checks = data.checks || {};
            const allOk = checks.kuzai_models_api && checks.darkai_models_api;
            setGlobalStatus(allOk);
            setBadge("badge-kuzai",  Boolean(checks.kuzai_models_api));
            setBadge("badge-darkai", Boolean(checks.darkai_models_api));
            setText("kuzai-model",   data.nodes?.kuzai?.model_name  || "--");
            setText("darkai-model",  data.nodes?.darkai?.model_name || "--");
            setText("kuzai-http",    String(data.nodes?.kuzai?.http_code  ?? "--"));
            setText("darkai-http",   String(data.nodes?.darkai?.http_code ?? "--"));
            setText("kuzai-latency",  (data.nodes?.kuzai?.latency_ms  != null) ?
data.nodes.kuzai.latency_ms  + " ms" : "--");
            setText("darkai-latency", (data.nodes?.darkai?.latency_ms != null) ?
data.nodes.darkai.latency_ms + " ms" : "--");
            setText("engine-status", allOk ? "ONLINE" : "DEGRADED");
            setText("web-status",    "ONLINE");
            setText("last-update",   data.app?.timestamp || "--");
        } catch (error) {
            setGlobalStatus(false);
            setBadge("badge-kuzai",  false);
            setBadge("badge-darkai", false);
            setText("engine-status", "ERROR");
        }
    }

    async function loadOrchestratorList() {
        try {
            const response = await fetch(cfg.orchestratorListUrl, { cache: "no-store" });
            const data = await response.json();
            if (!data.ok || !Array.isArray(data.items)) return [];
            const select = byId("orchestrator_select");
            if (select) {
                select.innerHTML = data.items.map(item =>
                    `<option
```

```
value="${escapeHtml(item.slug)}">${escapeHtml(item.name)}</option>`
                ).join('');
            }
            return data.items;
        } catch { return []; }
    }

    async function loadOrchestrator(slug) {
        try {
            const response = await fetch(cfg.orchestratorReadUrl + '?name=' +
encodeURIComponent(slug), { cache: "no-store" });
            const data = await response.json();
            if (!data.ok || !data.profile) throw new Error(data.error || "Load failed");
            fillProfileForm(data.profile);
            setText("profile-note", "Orchestrator '" + data.profile.name + "' loaded.");
        } catch (error) {
            setText("profile-note", "Load failed: " + error.message);
        }
    }

    async function saveOrchestrator(saveAs = false) {
        const profile = {
            slug: byId("profile_slug")?.value?.trim() || "",
            name: byId("profile_name")?.value?.trim() || "Default",
            description: byId("profile_description")?.value?.trim() || "",
            run: {
                turns:         parseInt(byId("profile_turns")?.value) || 6,
                max_lines:     parseInt(byId("profile_max_lines")?.value) || 5,
                max_chars:     parseInt(byId("profile_max_chars")?.value) || 500,
                history_depth: parseInt(byId("profile_history_depth")?.value) || 3,
            },
            kuzai: {
                system_prompt:  byId("kuzai_system_prompt")?.value  || "",
                temperature:    parseFloat(byId("kuzai_temperature")?.value)    || 0.35,
                top_p:          parseFloat(byId("kuzai_top_p")?.value)          || 0.95,
                top_k:          parseInt(byId("kuzai_top_k")?.value)            || 40,
                max_tokens:     parseInt(byId("kuzai_max_tokens")?.value)       || 300,
                repeat_penalty: parseFloat(byId("kuzai_repeat_penalty")?.value) || 1.05,
            },
            darkai: {
```

```
            system_prompt:  byId("darkai_system_prompt")?.value  || "",
            temperature:    parseFloat(byId("darkai_temperature")?.value)    || 0.35,
            top_p:          parseFloat(byId("darkai_top_p")?.value)          || 0.95,
            top_k:          parseInt(byId("darkai_top_k")?.value)            || 40,
            max_tokens:     parseInt(byId("darkai_max_tokens")?.value)       || 300,
            repeat_penalty: parseFloat(byId("darkai_repeat_penalty")?.value) || 1.05,
        },
    };

    let saveAsName = profile.slug;
    if (saveAs) {
        const input = prompt("Save as (slug name):", profile.slug || "");
        if (!input) return null;
        saveAsName = input.trim();
    }

    const response = await fetch(cfg.orchestratorSaveUrl, {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ profile, save_as_name: saveAsName })
    });

    const data = await response.json();
    if (!data.ok) throw new Error(data.error || "Save failed");

    currentOrchestratorSlug = data.slug;
    await loadOrchestratorList();
    const select = byId("orchestrator_select");
    if (select && data.slug) select.value = data.slug;
    setText("profile-note", "Saved as '" + data.slug + "'.");
    return data.profile || { slug: data.slug };
}

async function loadRunStatus() {
    try {
        const response = await fetch(cfg.runStatusUrl, { cache: "no-store" });
        const data = await response.json();
        const isRunning = Boolean(data.is_running);
        if (isRunning) runInterfaceReset = false;
        if (!isRunning && runInterfaceReset) {
```

```
                updateRunButtons(false, false, false);
                applyRunState("reset");
                return;
            }
            updateRunButtons(isRunning, false, false);
            applyRunState(data.status || "idle");
            setText("run-pid",     data.pid ? String(data.pid) : "--");
            setText("run-started", data.active?.started_at || "--");
            syncSelectedOrchestratorDisplay(data.active?.orchestrator_name ||
byId("orchestrator_select")?.value || "--");
            setText("run-note", data.note || "No run status available.");
            renderTranscript(data.transcript || [], data.stdout_tail || [], data.status ||
"idle", data.note || "");
        } catch (error) {
            updateRunButtons(false, false, false);
            applyRunState("error");
            setText("run-note", "Run status error: " + error.message);
            renderSystemTranscript("Run status error: " + error.message);
        }
    }

    async function startRun() {
        const openingPrompt = byId("opening_prompt")?.value?.trim() || "";
        if (!openingPrompt) { setText("run-note", "Opening prompt is required."); return; }
        runInterfaceReset = false;
        updateRunButtons(false, true, false);
        applyRunState("starting");
        setText("run-note", "Saving orchestrator and starting run...");
        renderSystemTranscript("Starting new run...");
        try {
            const savedProfile = await saveOrchestrator(false);
            if (!savedProfile || !savedProfile.slug) throw new Error("Unable to save
orchestrator.");
            const response = await fetch(cfg.runStartUrl, {
                method: "POST",
                headers: { "Content-Type": "application/json" },
                body: JSON.stringify({ opening_prompt: openingPrompt, orchestrator_name:
savedProfile.slug })
            });
            const data = await response.json();
```

```
            if (!response.ok || !data.ok) throw new Error(data.error || "HTTP " +
response.status);
            syncSelectedOrchestratorDisplay(data.orchestrator_name || savedProfile.slug);
            setText("run-note", "Run started — PID " + data.pid + ".");
            await loadRunStatus();
        } catch (error) {
            updateRunButtons(false, false, false);
            applyRunState("error");
            setText("run-note", "Start failed: " + error.message);
            renderSystemTranscript("Start failed: " + error.message);
        }
    }

    async function stopRun() {
        updateRunButtons(true, false, true);
        applyRunState("stopping");
        setText("run-note", "Stopping run...");
        try {
            const response = await fetch(cfg.runStopUrl, { method: "POST" });
            const data = await response.json();
            if (!response.ok || !data.ok) throw new Error(data.error || "HTTP " +
response.status);
            setText("run-note", data.message || "Run stopped.");
            await loadRunStatus();
        } catch (error) {
            updateRunButtons(false, false, false);
            applyRunState("error");
            setText("run-note", "Stop failed: " + error.message);
        }
    }

    function initEvents() {
        const loadBtn   = byId("orchestrator_load_btn");
        const saveBtn   = byId("orchestrator_save_btn");
        const saveAsBtn = byId("orchestrator_save_as_btn");
        const startBtn  = byId("start-run-btn");
        const stopBtn   = byId("stop-run-btn");
        const resetBtn  = byId("reset-run-btn");
        const select    = byId("orchestrator_select");
```

```
        if (loadBtn)   loadBtn.addEventListener("click",  async () => { const slug =
byId("orchestrator_select")?.value; if (slug) await loadOrchestrator(slug); });
        if (saveBtn)   saveBtn.addEventListener("click",  async () => { await
saveOrchestrator(false); });
        if (saveAsBtn) saveAsBtn.addEventListener("click",async () => { await
saveOrchestrator(true);  });
        if (startBtn)  startBtn.addEventListener("click", startRun);
        if (stopBtn)   stopBtn.addEventListener("click",  stopRun);
        if (resetBtn)  resetBtn.addEventListener("click", resetRunInterface);
        if (select) {
            select.addEventListener("change", async () => {
                const slug = select.value;
                if (slug) { await loadOrchestrator(slug);
syncSelectedOrchestratorDisplay(slug); }
            });
        }
    }

    async function init() {
        initEvents();
        const items = await loadOrchestratorList();
        if (items.length > 0) { await loadOrchestrator(items[0].slug);
syncSelectedOrchestratorDisplay(items[0].slug); }
        else { setText("profile-note", "No orchestrator profile found. Save the current editor
as a new profile."); }
        await loadStatus();
        await loadRunStatus();
        window.setInterval(() => { loadStatus(); loadRunStatus(); }, cfg.pollIntervalMs);
    }

    init().catch((error) => {
        setText("profile-note", "Initialization failed: " + error.message);
        setText("run-note",     "Initialization failed: " + error.message);
        applyRunState("error");
    });
})();
```

`public/assets/css/style.css`

Dark neon-blue theme built entirely with CSS custom properties. No framework dependency. Color ramps, border styles, and component states are all defined on `:root` and applied consistently across node cards, run state badges, and transcript message cards.

Dark neon-blue theme. CSS custom properties on `:root`. No framework dependency.

```
:root {
    --bg-1: #06162c;
    --bg-2: #0c2344;
    --bg-3: #12325f;
    --card-bg: rgba(17, 39, 74, 0.88);
    --card-bg-2: rgba(12, 30, 58, 0.94);
    --text-main: #ffffff;
    --text-soft: #eaf4ff;
    --text-dim: #b8d3f6;
    --neon-cyan: #7bf2ff;
    --neon-blue: #8fc4ff;
    --neon-violet: #c19cff;
    --neon-white: #fbfdff;
    --success: #67f3ba;
    --danger: #ff7b9d;
    --warning: #ffd97c;
    --info: #9fd6ff;
    --border-main: rgba(159, 214, 255, 0.26);
    --border-strong: rgba(123, 242, 255, 0.42);
    --shadow-main: 0 18px 50px rgba(0, 0, 0, 0.28);
    --shadow-neon: 0 0 18px rgba(123, 242, 255, 0.16);
    --radius-xl: 22px;
    --radius-lg: 16px;
    --radius-md: 12px;
}

/* ... full stylesheet — see public/assets/css/style.css */

html, body { margin: 0; padding: 0; min-height: 100%;
    font-family: "Segoe UI", "Inter", "Arial", sans-serif;
    background: radial-gradient(circle at top left, rgba(110, 180, 255, 0.24), transparent
28%),
        radial-gradient(circle at top right, rgba(165, 115, 255, 0.18), transparent 24%),
        linear-gradient(180deg, var(--bg-1) 0%, var(--bg-2) 46%, var(--bg-3) 100%);
    color: var(--text-main); }
```

```
.app-shell { width: min(1680px, calc(100% - 32px)); margin: 0 auto; padding: 24px 0 40px; }

.topbar { display: flex; justify-content: space-between; align-items: center; gap: 24px;
    margin-bottom: 24px; padding: 18px 22px;
    border: 1px solid rgba(159, 214, 255, 0.24); border-radius: var(--radius-xl);
    background: rgba(10, 29, 58, 0.9); box-shadow: var(--shadow-main); backdrop-filter:
blur(14px); }

.status-pill { display: inline-flex; align-items: center; gap: 10px; padding: 12px 16px;
    border-radius: 999px; border: 1px solid rgba(103, 243, 186, 0.38);
    background: rgba(16, 64, 53, 0.34); color: #e2fff5; font-weight: 700; }

.status-pill--down { border-color: rgba(255, 123, 157, 0.38); background: rgba(82, 18, 38,
0.34); color: #ffe7ee; }

.node-card, .panel, .editor-box { border: 1px solid var(--border-main); border-radius: var(--
radius-xl);
    background: var(--card-bg); box-shadow: var(--shadow-main); backdrop-filter: blur(12px);
padding: 22px; }

.node-card--kuzai  { border-color: rgba(123, 242, 255, 0.28); }
.node-card--darkai { border-color: rgba(193, 156, 255, 0.28); }

.node-badge--ok   { border-color: rgba(103, 243, 186, 0.36); color: #dffdf2; background:
rgba(16, 64, 53, 0.32); }
.node-badge--down { border-color: rgba(255, 123, 157, 0.36); color: #ffe7ee; background:
rgba(82, 18, 38, 0.32); }

.run-state-badge--running   { border-color: rgba(159, 214, 255, 0.36); background: rgba(26,
56, 92, 0.36); }
.run-state-badge--completed { border-color: rgba(103, 243, 186, 0.36); background: rgba(16,
64, 53, 0.32); }
.run-state-badge--stopped   { border-color: rgba(255, 217, 124, 0.38); background: rgba(83,
61, 13, 0.34); }
.run-state-badge--error     { border-color: rgba(255, 123, 157, 0.38); background: rgba(82,
18, 38, 0.34); }

.message-card--kuzai  { border-color: rgba(123, 242, 255, 0.24); }
.message-card--darkai { border-color: rgba(193, 156, 255, 0.24); }
```

```
.message-card__speaker { color: var(--neon-cyan); font-weight: 800; }
.message-card--darkai .message-card__speaker { color: var(--neon-violet); }

.btn--primary { background: linear-gradient(135deg, var(--neon-cyan), var(--neon-blue));
    color: #031124; box-shadow: 0 10px 28px rgba(123, 242, 255, 0.2); }
.btn:hover { transform: translateY(-1px); }
.btn:disabled { opacity: 0.55; cursor: not-allowed; transform: none; }

@media (max-width: 780px) {
    .topbar { flex-direction: column; }
    .two-col-grid, .hero-stats, .node-metrics { grid-template-columns: 1fr; }
    .btn { width: 100%; }
}
```

`SYSTEM DIRECTORY LAYOUT`

Complete reference of all paths used by the project across both nodes.
Covers llama.cpp binaries, model files, orchestrator scripts, web application files, persistent storage, systemd services, and CUDA installation.

```
/opt/src/llama.cpp/                              llama.cpp source tree
/opt/src/llama.cpp/build/bin/llama-server        inference server binary
/opt/src/llama.cpp/build/bin/llama-cli           CLI test binary

/opt/llm/models/                                 GGUF model files
/opt/llm/orchestrator/                           Python scripts
/opt/llm/orchestrator/venv/                      Python virtualenv
/opt/llm/orchestrator/runs/run-ab-YYYYMMDD/      session transcripts
/opt/llm/orchestrator/runs/run-ab-YYYYMMDD/transcript.json
/opt/llm/orchestrator/runs/run-ab-YYYYMMDD/transcript.md

/var/www/html/KUZCHAT-LLM-DUO/public/            PHP web application
/var/www/html/KUZCHAT-LLM-DUO/public/api/        PHP API endpoints
/var/www/html/KUZCHAT-LLM-DUO/public/assets/css/  style.css
/var/www/html/KUZCHAT-LLM-DUO/public/assets/js/   app.js
/var/www/html/KUZCHAT-LLM-DUO/app/config.php     central config
/var/www/html/KUZCHAT-LLM-DUO/storage/orchestrators/  profiles JSON
/var/www/html/KUZCHAT-LLM-DUO/storage/runs/      run transcripts
/var/www/html/KUZCHAT-LLM-DUO/storage/launcher/  PID + meta files
```

```
/var/log/llm-duo/                                service logs

/etc/systemd/system/llama-server-a.service       KUZAI service (fhc2)
/etc/systemd/system/llama-server-b.service       DARKAI service (fhc)

/usr/local/cuda/                                 CUDA toolkit
/etc/profile.d/cuda.sh                           PATH + LD_LIBRARY_PATH exports
```

`DEPLOYED MODEL`

Two distinct models are deployed, one per node.
The model choice on each node is constrained by available VRAM. Both use Q4_K_M quantization which offers a good balance between inference speed and output quality on consumer-grade GPUs.

Parameter	Value
Name	`bartowski_Mistral-7B-Instruct-v0.3-GGUF_Mistral-7B-Instruct-v0.3-Q4_K_M.gguf`
Family	Mistral 7B Instruct
Quantization	Q4_K_M
Prompt perf.	-
Generation perf.	-
Node	fhc2 (KUZAI) - RTX 5060 8 GB VRAM

Parameter	Value
Name	`bartowski/granite-3.1-3b-a800m-instruct-Q4_K_M.gguf`
Family	IBM Granite 3.1 - 3B parameters
Quantization	Q4_K_M
Prompt perf.	307 t/s
Generation perf.	136 t/s
Node	fhc (DARKAI) - RTX 3050 4 GB VRAM

`REPOSITORY DIRECTORY`

Complete list of all files and directories in the repository. Each entry maps directly to a component of the deployed system.

File	Description

`ORCHESTRATOR-01.py`	Generic orchestrator - all URLs required via CLI
`ORCHESTRATOR-02.py`	Production orchestrator - fhc/fhc2 IPs hardcoded as defaults
`kuzchat-llm-duo.conf`	Apache VirtualHost config
`MONITOR-FHC.sh`	Full Node B audit
`MONITOR-FHC2.sh`	Full Node A audit + Python venv check
`VERIF-CON-KUZAI-DARKAI.sh`	Inter-node connectivity and API test
`LLM-DUO-RUN-AWAY-#02.md`	Deployment journal phase 2
`LLM-DUO-RUN-AWAY-#03.md`	Deployment journal phase 3 - full Node B setup
`app/config.php`	Central application configuration
`public/index.php`	Main HTML shell
`public/api/status.php`	System + node health endpoint
`public/api/run-start.php`	Launch orchestrator process
`public/api/run-status.php`	Poll run state + transcript
`public/api/run-stop.php`	Stop running process
`public/api/orchestrators-list.php`	List saved profiles
`public/api/orchestrator-read.php`	Read profile by slug
`public/api/orchestrator-save.php`	Create or update profile
`public/assets/css/style.css`	Dark neon-blue theme
`public/assets/js/app.js`	Frontend controller
`storage/`	PHP persistent data

`KUSANAGI8200 - THE KUZ NETWORK - @2026`

KUZ LLM DUO - APPLICATION SYNOPTIC

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>KUZCHAT-LLM-DUO - Synoptic (Architecture Diagram V3)</title>
<style>
*{box-sizing:border-box;margin:0;padding:0}
:root{
  --bg:#06101e;
  --bg2:#08172e;
  --panel:#081829;
  --panel-2:#071520;
  --panel-3:#0b1020;
  --stroke:#17344c;
  --stroke-2:#234a68;
  --cyan:#7bf2ff;
  --cyan-soft:#7aafcc;
  --violet:#c19cff;
  --amber:#ffc966;
  --green:#67f3ba;
  --coral:#ff9b7a;
  --text:#d9ebf7;
  --muted:#8ea8b8;
  --micro:#4a7a9b;
  --shadow:0 12px 34px rgba(0,0,0,.30);
  --glow-cyan:0 0 0 1px rgba(123,242,255,.10),0 0 20px rgba(123,242,255,.08);
  --glow-violet:0 0 0 1px rgba(193,156,255,.10),0 0 20px rgba(193,156,255,.08);
  --glow-amber:0 0 0 1px rgba(255,201,102,.08),0 0 18px rgba(255,201,102,.06);
}
body{
  background:
    radial-gradient(circle at top left, rgba(123,242,255,.06), transparent 24%),
    radial-gradient(circle at top right, rgba(193,156,255,.06), transparent 24%),
```

```
    linear-gradient(180deg,var(--bg) 0%,var(--bg2) 100%);
  color:var(--text);
  font-family:'Courier New',monospace;
  min-height:100vh;
  padding:20px;
}
.wrapper{max-width:1620px;margin:0 auto}
.header{text-align:center;margin-bottom:20px}
.header h1{color:var(--cyan);font-size:18px;letter-spacing:.2em;text-transform:uppercase;margin-bottom:6px;font-weight:400}
.header p{color:var(--micro);font-size:12px;letter-spacing:.12em;text-transform:uppercase}
.synoptic{display:flex;flex-direction:column;gap:16px}
.top-flow{display:grid;grid-template-columns:repeat(4,minmax(0,1fr));gap:10px}
.flow-step{border:1px solid var(--stroke-2);background:linear-gradient(180deg,#081626 0%,#07111f 100%);border-radius:14px;padding:12px 14px;min-height:74px;box-shadow:var(--shadow);position:relative}
.flow-step:not(:last-child)::after{content:"→";position:absolute;right:-8px;top:50%;transform:translateY(-50%);color:var(--cyan-soft);font-size:18px;z-index:2}
.flow-kicker{color:var(--micro);font-size:9px;text-transform:uppercase;letter-spacing:.12em;margin-bottom:7px}
.flow-title{color:var(--cyan);font-size:11px;text-transform:uppercase;margin-bottom:6px}
.flow-text{color:var(--muted);font-size:10px;line-height:1.5;overflow-wrap:anywhere}
.link-rail{display:grid;grid-template-columns:repeat(4,minmax(0,1fr));gap:10px;align-items:center}
.rail-card{border:1px solid var(--stroke);background:rgba(8,20,34,.8);border-radius:12px;padding:10px 12px;min-height:58px;display:flex;flex-direction:column;gap:6px;justify-content:center}
.rail-title{font-size:10px;text-transform:uppercase;letter-spacing:.08em;color:var(--cyan-soft)}
.rail-text{font-size:9px;line-height:1.45;color:var(--muted)}
.rail-card.http{border-color:#2a6080}
.rail-card.poll{border-color:#2a6080;border-style:dashed}
.rail-card.exec{border-color:#a07020}
.rail-card.llm{border-color:#1a6a5a}
.band{border:1px dashed var(--stroke);border-radius:16px;padding:14px;background:rgba(6,16,30,.34);box-shadow:var(--shadow)}
.band.cyan{box-shadow:var(--shadow),var(--glow-cyan)}
.band.violet{box-shadow:var(--shadow),var(--glow-violet)}
.band.amber{box-shadow:var(--shadow),var(--glow-amber)}
.band-head{display:flex;justify-content:space-between;align-items:center;gap:12px;margin-bottom:12px;flex-wrap:wrap}
```

```
.band-title{color:var(--micro);font-size:11px;letter-spacing:.18em;text-transform:uppercase}
.band-note{color:var(--muted);font-size:10px;letter-spacing:.06em}
.grid{display:grid;grid-template-columns:repeat(12,minmax(0,1fr));gap:12px}
.card{background:linear-gradient(180deg,var(--panel) 0%,var(--panel-2) 100%);border:1px solid
var(--stroke-2);border-radius:12px;padding:14px;min-height:100px;display:flex;flex-
direction:column;gap:8px;overflow:hidden;box-shadow:var(--shadow)}
.card.cyan{border-color:#1a4a5a;box-shadow:var(--shadow),var(--glow-cyan)}
.card.violet{border-color:#3a2a5a;background:linear-gradient(180deg,#0b0d1d 0%,#0a0b18
100%);box-shadow:var(--shadow),var(--glow-violet)}
.card.amber{border-color:#5a4926;background:linear-gradient(180deg,#111018 0%,#0c0c13
100%);box-shadow:var(--shadow),var(--glow-amber)}
.span-2{grid-column:span 2}.span-3{grid-column:span 3}.span-4{grid-column:span 4}.span-5{grid-
column:span 5}.span-6{grid-column:span 6}.span-8{grid-column:span 8}.span-12{grid-column:span
12}
.card-title{font-size:11px;letter-spacing:.08em;text-transform:uppercase;color:var(--
cyan);font-weight:700}
.card.violet .card-title{color:var(--violet)}
.card.amber .card-title{color:var(--amber)}
.card-sub{font-size:10px;color:var(--cyan-soft);line-height:1.45}
.card-text{font-size:10px;color:var(--muted);line-height:1.55;overflow-wrap:anywhere;word-
break:break-word;white-space:normal}
.row{display:grid;grid-template-columns:repeat(auto-fit,minmax(120px,1fr));gap:8px}
.mini{border:1px solid #143047;background:#06101e;border-radius:8px;padding:10px;min-
height:54px}
.mini-label{font-size:9px;color:var(--micro);margin-bottom:6px;text-
transform:uppercase;letter-spacing:.08em}
.mini-value{font-size:10px;color:var(--text);line-height:1.4;overflow-wrap:anywhere}
.status-pill{display:inline-flex;align-items:center;gap:8px;width:max-content;max-
width:100%;padding:8px 12px;border-radius:999px;border:1px solid
#1b5c3a;background:#0a1f12;color:var(--green);font-size:10px}
.badges{display:flex;gap:8px;flex-wrap:wrap}.badge{display:inline-flex;align-
items:center;padding:6px 10px;border-radius:999px;font-size:9px;letter-spacing:.08em;text-
transform:uppercase;border:1px solid #17344c;background:#08131f;color:var(--muted)}
.badge.cyan{border-color:#1a4a5a;color:var(--cyan)}.badge.violet{border-
color:#3a2a5a;color:var(--violet)}.badge.amber{border-color:#5a4926;color:var(--
amber)}.badge.green{border-color:#1b5c3a;color:var(--green)}.badge.coral{border-
color:#5a2a1a;color:var(--coral)}
.dot{width:8px;height:8px;border-radius:50%;background:var(--green);flex:0 0 auto}
.transcript{display:grid;gap:10px}.msg{border:1px solid #17344c;background:#07111a;border-
radius:10px;padding:10px}.msg.violet{border-color:#352050;background:#0b0915}
.msg-meta{display:flex;justify-content:space-between;gap:10px;flex-wrap:wrap;margin-
```

```
bottom:6px}.msg-name{font-size:10px;color:var(--cyan);text-transform:uppercase;font-
weight:700}.msg.violet .msg-name{color:var(--violet)}.msg-time{font-size:9px;color:var(--
micro)}.msg-body{font-size:10px;line-height:1.6;color:var(--text);overflow-wrap:anywhere}
.arch-grid{display:grid;grid-template-columns:1.15fr .85fr;gap:14px}
.svg-panel{border:1px solid var(--stroke);border-radius:14px;background:linear-
gradient(180deg,#08131f 0%,#060d17 100%);padding:8px;overflow:auto}
svg{display:block;width:100%;height:auto;min-width:720px}
.legend{display:grid;grid-template-columns:repeat(auto-
fit,minmax(220px,1fr));gap:10px}.legend-item{border:1px solid #10283b;border-
radius:10px;padding:10px;background:#050d18;font-size:10px;color:var(--muted)}.legend-
line{display:inline-block;width:44px;height:0;border-top:2px solid #2a6080;margin-
right:8px;vertical-align:middle}.legend-line.dashed{border-top-style:dashed}.legend-
line.amber{border-color:#a07020}.legend-line.green{border-color:#1a6a5a}.legend-
line.violet{border-color:#404060}
@media (max-width:1280px){.arch-grid{grid-template-columns:1fr}.span-2,.span-3,.span-4,.span-
5,.span-6,.span-8{grid-column:span 6}.top-flow,.link-rail{grid-template-
columns:repeat(2,minmax(0,1fr))}.flow-step:nth-child(2)::after{content:''}}
@media (max-width:760px){body{padding:12px}.grid,.top-flow,.link-rail{grid-template-
columns:1fr}.span-2,.span-3,.span-4,.span-5,.span-6,.span-8,.span-12{grid-column:span
1}.header h1{font-size:14px;letter-spacing:.12em}.flow-step::after{display:none}}
</style>
</head>
<body>
<div class="wrapper">
  <header class="header">
    <h1>KUZCHAT-LLM-DUO - Application Synoptic</h1>
    <p>A KUZ NETWORK SOLUTION — [BETA-0.01.2026]</p>
  </header>

  <main class="synoptic">

    <section class="top-flow">
      <article class="flow-step"><div class="flow-kicker">Step 1</div><div class="flow-
title">Frontend</div><div class="flow-text">Browser loads the console, polls state and
transcript, and sends start or stop actions.</div></article>
      <article class="flow-step"><div class="flow-kicker">Step 2</div><div class="flow-
title">PHP Backend</div><div class="flow-text">PHP exposes API endpoints, validates profiles,
launches the orchestrator, and aggregates runtime status.</div></article>
      <article class="flow-step"><div class="flow-kicker">Step 3</div><div class="flow-
title">Python Orchestrator</div><div class="flow-text">The loop alternates KUZAI and DARKAI
turns, applies guardrails, and writes transcript artifacts.</div></article>
```

```
      <article class="flow-step"><div class="flow-kicker">Step 4</div><div class="flow-
title">Inference Nodes</div><div class="flow-text">llama-server on each node answers OpenAI-
compatible chat requests over port 8080.</div></article>
    </section>

    <section class="link-rail">
      <article class="rail-card http"><div class="rail-title">Frontend → Backend</div><div
class="rail-text">GET /, form POST, profile CRUD, stop action</div></article>
      <article class="rail-card poll"><div class="rail-title">Polling</div><div class="rail-
text">status.php and run-status.php every 5 seconds</div></article>
      <article class="rail-card exec"><div class="rail-title">Backend → Orchestrator</div><div
class="rail-text">nohup exec, PID file, launcher metadata</div></article>
      <article class="rail-card llm"><div class="rail-title">Orchestrator → Nodes</div><div
class="rail-text">POST /v1/chat/completions alternated for KUZAI and DARKAI</div></article>
    </section>

    <section class="band cyan">
      <div class="band-head">
        <div class="band-title">Browser / Frontend</div>
        <div class="band-note"></div>
      </div>
      <div class="grid">
        <article class="card cyan span-3"><div class="card-title">Topbar</div><div
class="card-sub">KUZCHAT LLM DUO · THE KUZ NETWORK · KUZAI.ORG</div><div class="status-
pill"><span class="dot"></span>READY / DEGRADED</div><div class="badges"><span class="badge
cyan">GET /</span><span class="badge green">online</span></div></article>
        <article class="card cyan span-5"><div class="card-title">Hero Card</div><div
class="card-sub">Duo orchestration console</div><div class="row"><div class="mini"><div
class="mini-label">Engine</div><div class="mini-value">ONLINE / DEGRADED</div></div><div
class="mini"><div class="mini-label">Web</div><div class="mini-value">ONLINE</div></div><div
class="mini"><div class="mini-label">Updated</div><div class="mini-value">2026-03-25
14:32</div></div></div></article>
        <article class="card cyan span-4"><div class="card-title">Infrastructure
Snapshot</div><div class="row"><div class="mini"><div class="mini-label">KUZAI</div><div
class="mini-value">fhc2 · 10.141.52.19 · HTTP 200 · 12 ms</div></div><div class="mini"><div
class="mini-label">DARKAI</div><div class="mini-value">fhc · 10.141.52.126 · HTTP 200 · 8
ms</div></div></div></article>
        <article class="card cyan span-6"><div class="card-title">Nodes Status</div><div
class="row"><div class="mini"><div class="mini-label">Node A · KUZAI</div><div class="mini-
value">Host: fhc2<br>API: :8080 /v1/chat/completions<br>Model: gemma-3-1b-it
Q4_K_M</div></div><div class="mini"><div class="mini-label">Node B · DARKAI</div><div
```

class="mini-value">Host: fhc
API: :8080 /v1/chat/completions
Model: granite-3.1-3b Q4_K_M </article>
        <article class="card violet span-6"> Orchestration Console Profile editor · params · run control Profile default / lab-01 / custom Turns 6 Max chars 500 History 3 loadsavestartstopreload </article>
        <article class="card violet span-12"> Live Transcript Polling run-status.php every 5 seconds KUZAI 2026-03-25 14:32:09 I'm KUZAI, running on fhc2. My role is to propose coherent on-premise architectures. DARKAI 2026-03-25 14:32:18 I'm DARKAI, on fhc. I audit KUZAI's proposals and ensure zero cloud drift. KUZAI 2026-03-25 14:32:27 For log aggregation I propose Promtail + Loki with local retention and no external dependency. </article>

    </section>

    <section class="band cyan">
      PHP Backend / Node A Apache 2 · PHP · 10.141.52.19:80

        <article class="card cyan span-4"> public/index.php HTML shell · PHP resolves config at render · injects window.KUZCHAT_CONFIG renderbootstrap </article>
        <article class="card cyan span-4"> app/config.php Nodes · IPs · endpoints · paths · UI params · shared by all PHP files shared config </article>
        <article class="card amber span-4"> Assets public/assets/js/app.js · polling · events · render
public/assets/css/style.css · dark neon-blue theme jscss </article>

```
        <article class="card span-3"><div class="card-title">status.php</div><div class="card-text">Reads /proc/meminfo · /proc/uptime · /proc/loadavg · probes both /v1/models endpoints · returns JSON</div></article>
        <article class="card span-3"><div class="card-title">run-start.php</div><div class="card-text">POST opening_prompt + orchestrator_name · validates paths · nohup exec · writes PID file</div></article>
        <article class="card span-3 violet"><div class="card-title">run-stop.php</div><div class="card-text">POST · SIGTERM · optional SIGKILL · removes PID file · writes stopped state</div></article>
        <article class="card span-3"><div class="card-title">run-status.php</div><div class="card-text">Checks /proc/PID · reads transcript.json · updates state machine · returns transcript entries</div></article>
        <article class="card violet span-4"><div class="card-title">orchestrators-list.php</div><div class="card-text">GET · returns sorted profiles from storage/orchestrators/*.json</div></article>
        <article class="card violet span-4"><div class="card-title">orchestrator-read.php</div><div class="card-text">GET ?name=slug · normalize + clamp fields · 404 if missing</div></article>
        <article class="card violet span-4"><div class="card-title">orchestrator-save.php</div><div class="card-text">POST profile + save_as_name · sanitize slug · normalize fields · write JSON</div></article>
        <article class="card amber span-4"><div class="card-title">storage/orchestrators/</div><div class="card-text">default.json · lab-01.json · custom profiles read/write by PHP orchestrator APIs</div></article>
        <article class="card amber span-4"><div class="card-title">storage/launcher/</div><div class="card-text">current_run.pid · current_run.json · stdout/stderr logs · liveness metadata written by PHP launcher</div></article>
        <article class="card amber span-4"><div class="card-title">storage/runs/</div><div class="card-text">run-ab-YYYYMMDD-HHMMSS · transcript.json · transcript.md written by the Python orchestrator output flow</div></article>
      </div>
    </section>

    <section class="band amber">
      <div class="band-head"><div class="band-title">Python Orchestrator</div><div class="band-note">/opt/llm/orchestrator/duo_loop_engine.py · virtualenv</div></div>
      <div class="grid">
        <article class="card amber span-5"><div class="card-title">Core Loop</div><div class="card-text">Turn 1 → KUZAI: build_prompt(opening, history=[])<br>Turn 2 → DARKAI: build_prompt(opening, history[-3:])<br>Alternating loop for N turns with normalized outputs</div><div class="badges"><span class="badge amber">A/B loop</span><span class="badge
```

amber">N turns </article>
        <article class="card amber span-3"> Constraints enforce_length() · max 4 sentences · 500 chars · strip_prefixes() · normalize_whitespace() </article>
        <article class="card amber span-4"> Outputs Writes to storage/runs/run-ab-*
transcript.json · transcript.md </article>
        <article class="card span-4"> CLI Parameters --turns 6 · --temperature-a 0.35 · --temperature-b 0.35 · --max-sentences 4 · --history-depth 3 · --max-chars 500 </article>
        <article class="card span-4"> Social Mode Detects prompts like introduce yourself / who are you / what is your name → short mode, reduced meta-commentary </article>
        <article class="card span-4"> Guardrails 100% local / on-prem scope · No cloud · No SaaS · No CDN · injected in every prompt </article>

    </section>

    <section class="band violet">
      Inference Nodes LAN 10.141.52.0/24 · llama.cpp + CUDA

        <article class="card cyan span-6"> Node A — fhc2 — KUZAI 10.141.52.19 · Intel i9 · 32 GB · NVIDIA RTX 5060 8 GB VRAM OS Ubuntu 24.04 CUDA 13.2 Driver nvidia-580 (hold) Kernel 6.17.0-19-generic llama-server-a.service · /opt/src/llama.cpp/build/bin/llama-server · host 0.0.0.0 · port 8080
Model: ggml-org/gemma-3-1b-it-GGUF:Q4_K_M · OpenAI-compatible API </article>
        <article class="card violet span-6"> Node B — fhc — DARKAI 10.141.52.126 · Intel i5 · 32 GB · NVIDIA RTX 3050 4 GB VRAM OS Debian Linux CUDA 12.9 Driver nvidia-575 (hold) Build b8357-89d0aec04 llama-server-b.service · /opt/src/llama.cpp/build/bin/llama-server · host 0.0.0.0 · port

8080
Model: bartowski/granite-3.1-3b-a800m-instruct-Q4_K_M.gguf · OpenAI-compatible API </article>
        <article class="card span-6"> Systemd Services /etc/systemd/system/llama-server-a.service → Node A / KUZAI
/etc/systemd/system/llama-server-b.service → Node B / DARKAI </article>
        <article class="card span-6"> Filesystem /opt/src/llama.cpp/build/bin/llama-server · inference binary
/opt/llm/models/ · GGUF files
/opt/llm/orchestrator/venv/ · Python virtualenv
/usr/local/cuda/ · /etc/profile.d/cuda.sh · CUDA + PATH </article>

    </section>

    <section class="arch-grid">
      <section class="band cyan">
        Architecture Flow Diagram

          <svg viewBox="0 0 980 430" xmlns="http://www.w3.org/2000/svg">
            <defs>
              <marker id="arrCyan" viewBox="0 0 10 10" refX="8" refY="5" markerWidth="5" markerHeight="5" orient="auto"><path d="M1 1 L9 5 L1 9" fill="none" stroke="#2a6080" stroke-width="1.6" fill="none"/></marker>
              <marker id="arrAmber" viewBox="0 0 10 10" refX="8" refY="5" markerWidth="5" markerHeight="5" orient="auto"><path d="M1 1 L9 5 L1 9" fill="none" stroke="#a07020" stroke-width="1.6" fill="none"/></marker>
              <marker id="arrGreen" viewBox="0 0 10 10" refX="8" refY="5" markerWidth="5" markerHeight="5" orient="auto"><path d="M1 1 L9 5 L1 9" fill="none" stroke="#1a6a5a" stroke-width="1.6" fill="none"/></marker>
            </defs>

            <rect x="20" y="30" width="180" height="110" rx="12" fill="#081829" stroke="#1a4a5a"/>
            <text x="35" y="55" fill="#7bf2ff" font-size="13">FRONTEND</text>
            <text x="35" y="75" fill="#8ea8b8" font-size="10">Browser UI</text>
            <text x="35" y="92" fill="#8ea8b8" font-size="10">Editor / Run</text>
            <text x="35" y="109" fill="#8ea8b8" font-size="10">Polling 5s</text>

            <rect x="260" y="30" width="220" height="130" rx="12" fill="#081829" stroke="#1a4a5a"/>
            <text x="275" y="55" fill="#7bf2ff" font-size="13">PHP BACKEND</text>
            <text x="275" y="75" fill="#8ea8b8" font-size="10">index.php / config.php</text>

```
            <text x="275" y="92" fill="#8ea8b8" font-size="10">run-start / run-stop</text>
            <text x="275" y="109" fill="#8ea8b8" font-size="10">status / run-status</text>
            <text x="275" y="126" fill="#8ea8b8" font-size="10">orchestrator APIs</text>

            <rect x="530" y="30" width="200" height="130" rx="12" fill="#111018"
stroke="#5a4926"/>
            <text x="545" y="55" fill="#ffc966" font-size="13">ORCHESTRATOR</text>
            <text x="545" y="75" fill="#8ea8b8" font-size="10">duo_loop_engine.py</text>
            <text x="545" y="92" fill="#8ea8b8" font-size="10">A/B loop</text>
            <text x="545" y="109" fill="#8ea8b8" font-size="10">guardrails</text>
            <text x="545" y="126" fill="#8ea8b8" font-size="10">transcripts</text>

            <rect x="780" y="30" width="170" height="130" rx="12" fill="#0b0d1d"
stroke="#3a2a5a"/>
            <text x="795" y="55" fill="#c19cff" font-size="13">LLM NODES</text>
            <text x="795" y="75" fill="#8ea8b8" font-size="10">KUZAI / DARKAI</text>
            <text x="795" y="92" fill="#8ea8b8" font-size="10">llama-server</text>
            <text x="795" y="109" fill="#8ea8b8" font-size="10">:8080 API</text>

            <rect x="230" y="230" width="220" height="110" rx="12" fill="#111018"
stroke="#5a4926"/>
            <text x="245" y="255" fill="#ffc966" font-size="13">STORAGE (PHP)</text>
            <text x="245" y="275" fill="#8ea8b8" font-size="10">orchestrators/</text>
            <text x="245" y="292" fill="#8ea8b8" font-size="10">launcher/</text>
            <text x="245" y="309" fill="#8ea8b8" font-size="10">pid / meta</text>

            <rect x="560" y="230" width="220" height="110" rx="12" fill="#111018"
stroke="#5a4926"/>
            <text x="575" y="255" fill="#ffc966" font-size="13">STORAGE (RUNS)</text>
            <text x="575" y="275" fill="#8ea8b8" font-size="10">runs/</text>
            <text x="575" y="292" fill="#8ea8b8" font-size="10">transcript.json</text>
            <text x="575" y="309" fill="#8ea8b8" font-size="10">transcript.md</text>

            <rect x="20" y="230" width="170" height="110" rx="12" fill="#081829"
stroke="#1a4a5a"/>
            <text x="35" y="255" fill="#7bf2ff" font-size="13">SYSTEM</text>
            <text x="35" y="275" fill="#8ea8b8" font-size="10">/proc</text>
            <text x="35" y="292" fill="#8ea8b8" font-size="10">/v1/models</text>
            <text x="35" y="309" fill="#8ea8b8" font-size="10">health</text>

            <path d="M200 80 L260 80" stroke="#2a6080" stroke-width="2" marker-
```

```
end="url(#arrCyan)" fill="none"/>
            <path d="M480 80 L530 80" stroke="#a07020" stroke-width="2" marker-
end="url(#arrAmber)" fill="none"/>
            <path d="M730 80 L780 80" stroke="#1a6a5a" stroke-width="2" marker-
end="url(#arrGreen)" fill="none"/>

            <path d="M370 160 L370 230" stroke="#a07020" stroke-width="2" stroke-dasharray="5
4" marker-end="url(#arrAmber)" fill="none"/>
            <path d="M630 160 L630 230" stroke="#1a6a5a" stroke-width="2" stroke-dasharray="5
4" marker-end="url(#arrGreen)" fill="none"/>
            <path d="M260 145 L220 145 L220 210 L190 210 L190 230" stroke="#2a6080" stroke-
width="2" stroke-dasharray="5 4" marker-end="url(#arrCyan)" fill="none"/>

            <text x="210" y="70" fill="#4a7a9b" font-size="9">HTTP</text>
            <text x="500" y="70" fill="#4a7a9b" font-size="9">EXEC</text>
            <text x="740" y="70" fill="#4a7a9b" font-size="9">LLM</text>




          </svg>
          <div class="note" style="margin-top:4px; padding:12px 14px; border:1px solid
rgba(122,198,255,.18); border-radius:10px; background:rgba(7,20,34,.45);">
            <div class="muted" style="line-height:1.7;">
              The browser interface sends HTTP requests to the PHP backend and polls it every
5 seconds for status updates.
              The PHP backend is the control plane: it starts and stops the Python
orchestrator, reads system metrics, checks
              the local and remote LLM APIs, and manages the PHP-side storage used for
orchestrator profiles, launcher files,
              PID tracking, and metadata. The Python orchestrator is the execution engine: it
runs the A/B loop, applies
              guardrails, queries the LLM nodes through llama-server endpoints, and writes run
outputs into the runs storage
              path. The system block represents monitoring inputs used by the PHP backend,
including /proc, /v1/models, and
              health checks. The frontend never talks directly to the LLM nodes.
            </div>
          </div>
        </div>
      </section>
```

```
      <section class="band violet">
        <div class="band-head"><div class="band-title">Execution Notes</div><div class="band-note"></div></div>
        <div class="grid">
          <article class="card cyan span-12"><div class="card-title">HTTP Sync Layer</div><div class="card-text">Initial page load, profile read/save, start and stop actions are synchronous exchanges between browser and PHP.</div></article>
          <article class="card cyan span-12"><div class="card-title">Polling Layer</div><div class="card-text">The frontend polls status.php and run-status.php every 5 seconds to refresh health data, run state, and transcript entries.</div></article>
          <article class="card amber span-12"><div class="card-title">Process Execution Layer</div><div class="card-text">run-start.php launches the Python orchestrator with nohup, then writes PID and launcher metadata used by subsequent status checks.</div></article>
          <article class="card amber span-12"><div class="card-title">LLM Exchange Layer</div><div class="card-text">The orchestrator alternates POST requests to each llama-server endpoint, applying prompt history rules and output constraints for every turn.</div></article>
          <article class="card violet span-12"><div class="card-title">Persistence Layer</div><div class="card-text">PHP storage keeps profiles and launcher metadata, while the Python orchestrator writes run folders and transcripts into the runs output path for traceability and replay.</div></article>
        </div>
      </section>
    </section>

    <section class="band">
      <div class="band-head"><div class="band-title">Legend</div><div class="band-note"></div></div>
      <div class="legend">
        <div class="legend-item"><span class="legend-line"></span>HTTP request / sync exchange</div>
        <div class="legend-item"><span class="legend-line dashed"></span>Polling / async fetch every 5 s</div>
        <div class="legend-item"><span class="legend-line amber"></span>Process launch / nohup exec</div>
        <div class="legend-item"><span class="legend-line green"></span>LLM inference POST /v1/chat/completions</div>
        <div class="legend-item"><span class="legend-line dashed green"></span>Storage read / write</div>
        <div class="legend-item"><span class="legend-line violet"></span>State / loop relation</div>
```

```
      </div>
    </section>

   </main>
</div>
</body>
</html>
```